

Team Notebook

December 17, 2025

Contents

1 DP	2	3 Flow	6	6.2 Diophantile	17
1.1 DC DP optimization	2	3.1 Dinic	6	6.3 FFT	18
1.2 sos	2	3.2 edmondkarp	7	6.4 LSieveBigModTernaryS	18
2 DS	2	3.3 mcmf	7	6.5 Miller Rabin	18
2.1 2D SPARSE TABLE	2	4 Geometry	8	6.6 matexpo	19
2.2 Centroid Decom	2	4.1 Convex hull	8	6.7 mobius	19
2.3 HASHING	3	4.2 Geo 2	8	6.8 ncr	19
2.4 HLD and Merge sort tree	3	4.3 Nearest point	8	6.9 stirling _n umber	20
2.5 LCA	4	4.4 geo _{2d}	8	6.10 totient	20
2.6 LINE EQ	4	5 Graph	15	7 String	20
2.7 MOs	4	5.1 Articulation Point _{Bridge}	15	7.1 AhoCorasick	20
2.8 Persistence trie	4	5.2 Bell	15	7.2 Manacher	21
2.9 Trie	5	5.3 euler _{path_directed}	16	7.3 palindromitree	21
2.10 persistentSegment	5	5.4 kosaraju	16	7.4 suffixArray	22
2.11 randomnumber	5	5.5 mos on trees	16	7.5 z kmp	22
2.12 xor _{basis}	6	5.6 roll _{dsu}	17	8 sublime build	22
		6 Number Theory Math	17	9 template	22
		6.1 CRT	17		

1 DP

1.1 DC DP optimization

```
/**
Let opt(i, j) be the value of k that minimizes the
above expression. Assuming that the cost function
satisfies the quadrangle inequality, we can show that
opt(i, j) <= opt(i, j + 1) for all
i, j. This is known as the monotonicity condition. Then,
we can apply divide and conquer DP. The optimal "
splitting point" for a fixed i increases as j increases
.
*/
int const N = 3e5 + 5, inf = 1e16;
int m, n, R = 0, L = 1, res = 0;
int ar[N], cnt[N];
vector<long long> dp_before, dp_cur;
void add(int id) {
    res += cnt[ar[id]]; cnt[ar[id]]++;
}
void Remove(int id) {
    cnt[ar[id]]--; res -= cnt[ar[id]];
}
long long C(int l, int r) {
    while(R < r) add(++R);
    while(L > l) add(--L);
    while(L < l) Remove(L++);
    while(R > r) Remove(R--);
    return res;
}
void compute(int l, int r, int optl, int optr) {
    if (l > r) return;
    int mid = (l + r) >> 1;
    pair<long long, int> best = {LLONG_MAX, -1};
    for (int k = optl; k <= min(mid, optr); k++) best = min(
        best, {(k ? dp_before[k - 1] : 0) + C(k, mid), k});
    dp_cur[mid] = best.first;
    int opt = best.second;
    compute(l, mid - 1, optl, opt); compute(mid + 1, r, opt,
        optr);
}
void solve() {
    cin >> n >> m;
    for (int i = 1; i <= n; i++) cin >> ar[i];
    dp_before.assign(n + 1, 0); dp_cur.assign(n + 1, 0);
    for (int i = 1; i <= n; i++) dp_before[i] = C(1, i);
    for (int i = 1; i < m; i++) {
        compute(1, n, 1, n); dp_before = dp_cur;
    }
}
```

```
cout << dp_before[n] << endl;
}
```

1.2 sos

```
const int B = 20;
int a[1 << B], f[1 << B], g[1 << B];
void solve() {
    int n; cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> a[i];
        f[a[i]]++;
        g[a[i]]++;
    }
    // sum over subsets
    for (int i = 0; i < B; i++) {
        for (int mask = 0; mask < (1 << B); mask++) {
            if ((mask & (1 << i)) != 0) {
                f[mask] += f[mask ^ (1 << i)];
            }
        }
    }
    // sum over supersets
    for (int i = 0; i < B; i++) {
        for (int mask = (1 << B) - 1; mask >= 0; mask--) {
            if ((mask & (1 << i)) == 0) g[mask] += g[mask ^ (1 << i)];
        }
    }
    for (int i = 1; i <= n; i++) {
        cout << f[a[i]] << ' ' << g[a[i]] << ' ' << n - f[(1 << B)
            ] - 1 - a[i] << '\n';
    }
}
```

2 DS

2.1 2DSPARSE TABLE

```
const ll N=1000; const ll M = 10; ll tab[M+1][M+1][N+1][N
+1]; ll L[N+1]; ll a[N+1][N+1]; ll s[N+1][N+1];
struct st2{ ll n,m; void init(ll _n,ll _m){ n=_n; m=_m; for(
ll i=2;i<=N;i++){ L[i]=L[i/2]+1; } } ll g(ll x,ll y,ll
z=0,ll w=0){ return max({x,y,z,w}); } void build(){ for
(ll i=0;i<N;i++){ for(ll j=0;j<M;j++){ tab[0][0][i][j]=
a[i][j]; } } for(ll y=1;y<=M;y++){ for(ll i=0;i<N;i++){
```

```
for(ll j=0;j<M;j++){ if(j+(1<<y)-1<m) tab[0][y][i][j]=
g(tab[0][y-1][i][j],tab[0][y-1][i][j+(1<<(y-1))]); } }
} for(ll x=1;x<=M;x++){ for(ll i=0;i<N;i++){ for(ll j
=0;j<M;j++){ if(i+(1<<x)-1<n) tab[x][0][i][j]=g(tab[x
-1][0][i][j],tab[x-1][0][i+(1<<(x-1))][j]); } } } for(
ll x=1;x<=M;x++){ for(ll y=1;y<=M;y++){ for(ll i=0;i<N;
i++){ for(ll j=0;j<M;j++){ if(i+(1<<x)-1<n && j+(1<<y)
-1<m){ tab[x][y][i][j]=g( tab[x-1][y-1][i][j], tab[x
-1][y-1][i+(1<<(x-1))][j], tab[x-1][y-1][i][j+(1<<(y-1)
)], tab[x-1][y-1][i+(1<<(x-1))][j+(1<<(y-1))] ); } } }
} } ll qry_i(ll x,ll y,ll _x,ll _y){ ll lx=L[_x-x+1];
ll ly=L[_y-y+1]; return g( tab[lx][ly][x][y], tab[lx][
ly][_x-(1<<lx)+1][y], tab[lx][ly][x][_y-(1<<(ly))+1],
tab[lx][ly][_x-(1<<lx)+1][_y-(1<<(ly))+1] ); } };
```

2.2 Centroid Decom

```
struct node {
    vector<int> adj; int sub = 1; int p = -1; // centroid
    parent
    int dep = 0; int vis = 0; // removed (centroid)
};
node tree[N]; int n; ll ans = 0; int root;
/* ----- SUBTREE SIZE ----- */
void dfs(int u, int p) {
    tree[u].sub = 1;
    for (int v : tree[u].adj) {
        if (v != p && tree[v].vis == 0) {
            dfs(v, u);
            tree[u].sub += tree[v].sub;
        }
    }
}
/* ----- FIND CENTROID ----- */
int dfs2(int u, int p, int tot) {
    for (int v : tree[u].adj) {
        if (v != p && tree[v].vis == 0 && tree[v].sub > tot /
            2) {
            return dfs2(v, u, tot);
        }
    }
    return u;
}
/* ----- COLLECT LEVEL INFO (OPTIONAL) ----- */
/* Keep this if you need depth / distance info */
void nextlev(int u, int p, int d) {
    tree[u].dep = d;
    for (int v : tree[u].adj) {
```

```

    if (v != p && tree[v].vis == 0) {
        nextlev(v, u, d + 1);
    }
}
}
/* ----- SOLVE FOR A CENTROID ----- */
// void solve(int c) {
//     // example placeholder // do calculations using
//     centroid c
// }
/* ----- BUILD DECOMPOSITION ----- */
void build(int u, int p) {
    dfs(u, -1);
    int c = dfs2(u, -1, tree[u].sub);
    tree[c].p = p;
    if (p == -1) root = c;
    // optional: gather depth info from centroid
    nextlev(c, -1, 0);
    solve(c);
    tree[c].vis = 1;
    for (int v : tree[c].adj) {
        if (tree[v].vis == 0) {
            build(v, c);
        }
    }
}
// Build the tree inside main and call build(1, -1);

```

2.3 HASHING

```

int power(long long n, long long k, const int mod) { int ans
    = 1 % mod; n %= mod; if (n < 0) n += mod; while (k) {
    if (k & 1) ans = (long long) ans * n % mod; n = (long
    long) n * n % mod; k >>= 1; } return ans; } const int
MOD1 = 127657753, MOD2 = 987654319; const int p1 = 137,
p2 = 277; int ip1, ip2; pair<int, int> pw[N], ipw[N];
void prec() { pw[0] = {1, 1}; for (int i = 1; i < N; i
++) { pw[i].first = 1LL * pw[i - 1].first * p1 % MOD1;
pw[i].second = 1LL * pw[i - 1].second * p2 % MOD2; }
ip1 = power(p1, MOD1 - 2, MOD1); ip2 = power(p2, MOD2 -
2, MOD2); ipw[0] = {1, 1}; for (int i = 1; i < N; i++)
{ ipw[i].first = 1LL * ipw[i - 1].first * ip1 % MOD1;
ipw[i].second = 1LL * ipw[i - 1].second * ip2 % MOD2; }
} struct Hashing { int n; string s; // 0 - indexed
vector<pair<int, int>> hs; // 1 - indexed // Hashing()
{} void init(string _s) { n = _s.size(); s = _s; hs.
emplace_back(0, 0); for (int i = 0; i < n; i++) { pair<
int, int> p; p.first = (hs[i].first + 1LL * pw[i].first
* s[i] % MOD1) % MOD1; p.second = (hs[i].second + 1LL

```

```

* pw[i].second * s[i] % MOD2) % MOD2; hs.push_back(p);
} } pair<int, int> get_hash(int l, int r) { // 1 -
indexed assert(1 <= l && l <= r && r <= n); pair<int,
int> ans; ans.first = (hs[r].first - hs[l - 1].first +
MOD1) * 1LL * ipw[l - 1].first % MOD1; ans.second = (hs
[r].second - hs[l - 1].second + MOD2) * 1LL * ipw[l -
1].second % MOD2; return ans; } pair<int, int> get_hash
() { return get_hash(1, n); } };

```

2.4 HLD and Merge sort tree

```

typedef struct node {
    int p;
    vector<int> adj;
    int mxchild; int chain; int ind_in_chain; int subtree;
    int koto; int price;
} node;
node tree[200010];
vector<int> chain(200010);
vector<int> start_c(200010);
vector<vector<int>> seg(800010);
vector<vector<ll>> sum(800010);
vector<int> dfs_array(200010);
// vector<vector<seg>> segtree(500010);
int c;
int ind=-1;
void dfs(int u, int p) {
    tree[u].p = p;
    int mx=0;
    f(i, tree[u].adj.size()) {
        int v = tree[u].adj[i];
        if(p==v) continue;
        dfs(v,u);
        tree[u].subtree += tree[v].subtree;
        if(mx < tree[v].subtree) {mx = tree[v].subtree; tree[
            u].mxchild = v;}
    }
}
void hld(int u) {
    dfs_array[++ind] = u;
    chain[c]++;
    // chain[c].push_back(u);
    tree[u].chain = c;
    tree[u].ind_in_chain = chain[c] - 1;
    if(tree[u].mxchild) hld(tree[u].mxchild);
    f(i, tree[u].adj.size()) {
        int v = tree[u].adj[i];
        if(v == tree[u].p || v == tree[u].mxchild) continue;
        c++;
    }
}

```

```

    start_c[c] = ind+1;
    hld(v);
}
}
void merge(int ind) {
    int lsize = seg[2*ind+1].size();
    int rsize = seg[2*ind+2].size();
    seg[ind].clear();
    sum[ind].clear();
    seg[ind].resize(lsize+rsize);
    sum[ind].resize(lsize+rsize);
    int l = 0, r = 0, i=0;
    while(l < lsize || r < rsize) {
        if(l==lsize) {
            seg[ind][i] =(seg[2*ind+2][r]);r++;
        }
        else if(r==rsize) {
            seg[ind][i] =(seg[2*ind+1][l]);l++;
        }
        else if(seg[2*ind+1][l] < seg[2*ind+2][r]) {
            seg[ind][i] =(seg[2*ind+1][l]);l++;
        }
        else {
            seg[ind][i] =(seg[2*ind+2][r]);r++;
        }
        sum[ind][i] = ((l==0 ? 0 : sum[ind*2+1][l-1]) + (r==0
            ? 0 : sum[ind*2+2][r-1]));
        i++;
    }
    // return narr;
}
void build(vector<int>& arr, int l, int r, int ind) {
    if(l==r) {
        l = arr[l];
        seg[ind].resize(1);
        sum[ind].resize(1);
        seg[ind][0] = (tree[l].price);
        sum[ind][0] = ((ll)tree[l].koto);
    } else {
        int m = (l+r)/2;
        build(arr,l,m,2*ind+1); build(arr,m+1,r,2*ind+2);
        merge(ind);
    }
}
ll query(int s, int e, int l, int r, int ind, int tar) {
    if(s>e) return 0;
    if(s==l && e==r) {
        int ans = -1;
    }
}

```

```

l = 0, r = seg[ind].size()-1;
while(l <= r) {
    int m = (l+r)/2;
    if(seg[ind][m] <= tar)
        ans = m, l = m+1;
    else r = m-1;
}
return ans== -1?0:sum[ind][ans];
} else {
    int m = (l+r)/2;
    return query(s,min(e,m),l,m,2*ind+1,tar) + query(max(s,m+1),e,m+1,r,2*ind+2,tar);
}
}
}

```

2.5 LCA

```

typedef struct node {
    int val;vector<int> adj;vector<int> p;vector<int> ele;
    int childv;int dep;
} node;
node tree[100005];
void dfs(int u, int p) {
    tree[u].p[0] = p;
    tree[u].dep = tree[p].dep + 1;
    for(int i = 1; i <= (int)log2(n) + 1; i++) {
        if(tree[u].p[i-1] == 0) break;
        tree[u].p[i] = tree[tree[u].p[i-1]].p[i-1];
    }
    for(int i = 0; i < tree[u].adj.size(); i++) if(tree[u].adj[i] != p) dfs(tree[u].adj[i],u);
}
int lca(int u, int v) {
    if(tree[v].dep < tree[u].dep) swap(v,u);
    for(int i = (int)log2(n) + 1; i >= 0; i--) {
        if(tree[v].p[i] != tree[u].p[i]) {
            v = tree[v].p[i]; u = tree[u].p[i];
        }
    }
    if(v==u) return v;
    for(int i = (int)log2(n) + 1; i >= 0; i--) {
        if(tree[v].p[i] != tree[u].p[i]) {
            v = tree[v].p[i]; u = tree[u].p[i];
        }
    }
    return tree[v].p[0];
}

```

2.6 LINE EQ

```

//all solution of ax+by=c in [minx,maxx], [miny,maxy]
int gcd(int a, int b, int &x, int &y) { if (b == 0) { x = 1;
    y = 0; return a; } int x1, y1; int d = gcd(b, a % b,
    x1, y1); x = y1; y = x1 - y1 * (a / b); return d; }
bool solve_any(int a, int b, int c, int &x, int &y, int
    &g) { if (a == 0 && b == 0) { g = x = y = 0; return
    true; } g = gcd(abs(a), abs(b), x, y); if (c % g != 0)
    { return false; } x *= (c / g); y *= (c / g); if (a <
    0) x = -x; if (b < 0) y = -y; return true; } void shift
    (int &x, int &y, int a, int b, int cnt) { x += cnt * b;
    y -= cnt * a; } int solve_all(int a, int b, int c, int
    minx, int maxx, int miny, int maxy) { int x, y, g; if
    (!solve_any(a, b, c, x, y, g)) return 0; a /= g; b /= g
    ; int sign_a = a > 0 ? +1 : -1; int sign_b = b > 0 ? +1
    : -1; shift(x, y, a, b, (minx - x) / b); if (x < minx)
    { shift(x, y, a, b, sign_b); } if (x > maxx) return 0;
    int lx1 = x; shift(x, y, a, b, (maxx - x) / b); if (x
    > maxx) { shift(x, y, a, b, -sign_b); } int rx1 = x;
    shift(x, y, a, b, (y - miny) / a); if (y < miny) {
    shift(x, y, a, b, -sign_a); } if (y > maxy) return 0;
    int lx2 = x; shift(x, y, a, b, sign_a); } int rx2 = x; if (
    lx2 > rx2) swap(lx2, rx2); int lx = max(lx1, lx2); int
    rx = min(rx1, rx2); if (lx > rx) return 0; return (rx -
    lx) / abs(b) + 1; } void solve() { int a, b, c; cin >>
    a >> b >> c; int minx, miny, maxx, maxy; cin >> minx
    >> maxx >> miny >> maxy; if (maxx < minx || maxy < miny
    ) { cout << 0 << endl; exit(0); } if (a == 0 && b == 0)
    { if (c == 0) { cout << (maxy - miny + 1) * (maxx -
    minx + 1) << endl; } else { cout << 0 << endl; } } else
    if (a == 0 || b == 0) { if (a == 0) { if (c % b == 0
    && (c / b) >= miny && (c / b) <= maxy) { cout << maxx -
    minx + 1 << endl; } else { cout << 0 << endl; } } else
    { if (c % a == 0 && (c / a) >= minx && (c / a) <= maxx
    ) { cout << maxx - miny + 1 << endl; } else { cout << 0
    << endl; } } } else cout << solve_all(a, b, c, minx,
    maxx, miny, maxy) << endl; }

```

2.7 MOs

```

const int BLOCK_SIZE = 350;
int n, q, ar[N], freq[N], res = 0, ans[10*N];
struct Query {
    int l, r, id;
    bool operator<(const Query &y) const {
        int x_block = l / BLOCK_SIZE;
        int y_block = y.l / BLOCK_SIZE;

```

```

        if(x_block == y_block)
            return r < y.r;
        return x_block < y_block;
    }
};
void add(int i) {
    int x = ar[i]; res -= freq[x] / 2; freq[x]++; res += freq
        [x] / 2;
}
void Remove(int i) {
    int x = ar[i]; res -= freq[x] / 2; freq[x]--; res += freq
        [x] / 2;
}
void solve() { // need to reset freq if needed
    cin >> n;
    for (int i = 1; i <= n; i++) cin >> ar[i];
    cin >> q;
    vector<Query> qr;
    res = 0; int ii = 0;
    while(q--) {
        int l, r; cin >> l >> r; qr.push_back(Query({l, r, ii
            ++}));
    }
    sort(qr.begin(), qr.end());
    int left = 1, right = 0;
    for (Query q : qr) {
        int l = q.l, r = q.r;
        while(right < r) add(++right);
        while(left > l) add(--left);
        while(left < l) Remove(left++);

        while(right > r) Remove(right--);
        ans[q.id] = res;
    }
    for (int i = 0; i < ii; i++) cout << ans[i] << "\n";
}

```

2.8 Persistence trie

```

map<ll,ll>root; struct node { ll left = 0, right = 0; ll cnt
    = 0; }; node a[9000001]; ll ptr=2; struct p_trie{
    p_trie() { root[0] = 1; } void insert(ll par_id,ll
    curr_id,ll k){ root[curr_id]=ptr++; ll par_node=root[
    par_id]; ll curr_node=root[curr_id]; for(ll i=30;i>=0;i
    --){ if((k&(1ll<<i))) { a[curr_node].left = a[par_node
    ].left; a[curr_node].right = ptr++; curr_node = a[
    curr_node].right; par_node = a[par_node].right; }else{
    a[curr_node].right=a[par_node].right; a[curr_node].left
    =ptr++; curr_node=a[curr_node].left; par_node=a[

```

```

par_node].left; } a[curr_node].cnt=a[par_node].cnt+1; }
} ll max_xor(ll id,ll k){ ll curr=root[id]; ll ans=0;
for(ll i=30;i>=0;i--){ if((k&(1ll<<i))){ if(a[curr].
left){ ans|=(1ll<<i); curr=a[curr].left; }else{ curr=a[
curr].right; } }else{ if(a[curr].right){ ans|=(1ll<<i);
curr=a[curr].right; }else{ curr=a[curr].left; } } }
return ans; } ll min_xor(ll id,ll k){ ll curr=root[id];
ll ans=0; for(ll i=30;i>=0;i--){ if((k&(1ll<<i))){ if(
a[curr].right){ curr=a[curr].right; }else{ ans|=(1ll<<i
); curr=a[curr].left; } }else{ if(a[curr].left){ curr=a
[curr].left; }else{ ans|=(1ll<<i); curr=a[curr].right;
} } } return ans; } } }

```

2.9 Trie

```

const int N = 3e5 + 9;
struct Trie {
    static const int B = 31;
    struct node {
        node* nxt[2];
        int sz;
        node() {
            nxt[0] = nxt[1] = NULL;
            sz = 0;
        }
    }*root;
    Trie() {
        root = new node();
    }
    void insert(int val) {
        node* cur = root;
        cur->sz++;
        for (int i = B - 1; i >= 0; i--) {
            int b = val >> i & 1;
            if (cur->nxt[b] == NULL) cur->nxt[b] = new node();
            cur = cur->nxt[b];
            cur->sz++;
        }
    }
    int query(int x, int k) { // number of values s.t. val ^ x
        < k
        node* cur = root;
        int ans = 0;
        for (int i = B - 1; i >= 0; i--) {
            if (cur == NULL) break;
            int b1 = x >> i & 1, b2 = k >> i & 1;
            if (b2 == 1) {
                if (cur->nxt[b1]) ans += cur->nxt[b1]->sz;
                cur = cur->nxt[b1];
            }
        }
    }
}

```

```

    } else cur = cur->nxt[b1];
}
return ans;
}
int get_max(int x) { // returns maximum of val ^ x
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
        int k = x >> i & 1;
        if (cur->nxt[!k]) cur = cur->nxt[!k], ans <+= 1,
            ans++;
        else cur = cur->nxt[k], ans <+= 1;
    }
    return ans;
}
int get_min(int x) { // returns minimum of val ^ x
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
        int k = x >> i & 1;
        if (cur->nxt[k]) cur = cur->nxt[k], ans <+= 1;
        else cur = cur->nxt[!k], ans <+= 1, ans++;
    }
    return ans;
}
void del(node* cur) {
    for (int i = 0; i < 2; i++) if (cur->nxt[i]) del(cur->
        nxt[i]);
    delete(cur);
}
} t;

```

2.10 persistentSegment

```

struct node {
    ll sum; node *left, *right;
    node(ll val = 0) {
        sum = val; left = right = NULL;
    }
    node(node *l, node *r) {
        left = l; right = r; sum = (l ? l->sum : 0) + (r ? r
            ->sum : 0);
    }
};
int n; vector<int> v;
node* build(int start, int end) {
    if (start == end)
        return new node(v[start]);
    int mid = (start + end) >> 1;
}

```

```

        return new node(build(start, mid), build(mid + 1, end));
    }
    node* update(node* prev, int pos, ll val, int start, int end
        ) {
        if (start == end) return new node(val);
        int mid = (start + end) >> 1;
        if (pos <= mid) return new node(update(prev->left, pos,
            val, start, mid), prev->right);
        else return new node(prev->left, update(prev->right, pos,
            val, mid + 1, end));
    }
    ll query(node* cur, int l, int r, int start, int end) {
        if (cur == NULL || start > r || end < l) return 0;
        if (l <= start && end <= r) return cur->sum;
        int mid = (start + end) >> 1;
        return query(cur->left, l, r, start, mid) + query(cur->
            right, l, r, mid + 1, end);
    }
    void solve() {
        cin >> n; v.resize(n);
        for (int &x : v) cin >> x;
        vector<node*> roots; roots.push_back(build(0, n - 1));
        int q; cin >> q;
        while (q--) {
            int id; cin >> id;
            if (id == 1) {
                int ver, pos; ll val; cin >> ver >> pos >> val;
                roots.push_back(update(roots[ver], pos - 1, val,
                    0, n - 1));
            } else {
                int ver, l, r; cin >> ver >> l >> r;
                cout << query(roots[ver], l - 1, r - 1, 0, n - 1)
                    << "\n";
            }
        }
    }
}

```

2.11 randomnumber

```

struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::
            steady_clock::now().time_since_epoch().count();
    }
}

```

```
return splitmix64(x + FIXED_RANDOM);}); //For long long int
to long long int
```

2.12 xor_{basis}

```
struct XorBasis {
    vector<ll> basis;
    ll N = 0, tmp = 0;
    void add(ll x) {
        N++;
        tmp ^= x;
        for (ll b : basis) x = min(x, x ^ b);
        if (!x) return;
        for (ll &b : basis) if ((b ^ x) < b) b ^= x;
        basis.push_back(x);
        sort(basis.rbegin(), basis.rend());
    }
    // number of basis vectors (rank)
    ll size() { return (ll) basis.size(); }
    // clear everything
    void clear() { N = 0; tmp = 0; basis.clear(); }
    // returns true if x can be formed
    bool possible(ll x) {
        for (ll b : basis) x = min(x, x ^ b);
        return x == 0;
    }
    // returns maximum xor value
    ll maxxor(ll x = 0) {
        for (ll b : basis) x = max(x, x ^ b);
        return x;
    }
    // returns minimum xor value
    ll minxor(ll x = 0) {
        for (ll b : basis) x = min(x, x ^ b);
        return x;
    }
    // returns count of subsets which xor to x (modded)
    ll cntxor(ll x) {
        if (!possible(x)) return 0LL;
        ll freebits = N - size();
        ll res = 1;
        while (freebits--) res = (res * 2) % MOD;
        return res;
    }
    // returns sum of xor of all subsets
    ll sumOfAll() {
        return tmp * (1LL << (N - 1));
    }
    // returns k-th smallest xor value (1-indexed)
```

```
ll kth(ll k) {
    ll sz = size();
    if (k > (1LL << sz)) return -1;
    k--;
    ll ans = 0;
    for (ll i = 0; i < sz; i++)
        if (k >> (sz - 1 - i) & 1) ans ^= basis[i];
    return ans;
}
} xb;
```

3 Flow

3.1 Dinic

```
struct FlowEdge {
    int v, u;
    int cap, flow = 0;
    FlowEdge(int v, int u, int cap) : v(v), u(u), cap(cap) {}
};
struct Dinic {
    const int flow_inf = 1e8;
    vector<FlowEdge> edges;
    vector<vector<int>> adj;
    int n, m = 0;
    int s, t;
    vector<int> level, ptr;
    queue<int> q;
    Dinic(int n, int s, int t) : n(n), s(s), t(t) {
        adj.resize(n);
        level.resize(n);
        ptr.resize(n);
    }
    void add_edge(int v, int u, int cap) {
        edges.emplace_back(v, u, cap);
        edges.emplace_back(u, v, cap); // cap if undirected,
        otherwise 0;
        adj[v].push_back(m);
        adj[u].push_back(m + 1);
        m += 2;
    }
    bool bfs() {
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                if (edges[id].cap == edges[id].flow)
                    continue;
```

```
                if (level[edges[id].u] != -1)
                    continue;
                level[edges[id].u] = level[v] + 1;
                q.push(edges[id].u);
            }
        }
        return level[t] != -1;
    }
    int dfs(int v, int pushed) {
        if (pushed == 0) return 0;
        if (v == t) return pushed;
        for (int& cid = ptr[v]; cid < (int)adj[v].size(); cid
            ++){
            int id = adj[v][cid];
            int u = edges[id].u;
            if (level[v] + 1 != level[u]) continue;
            int tr = dfs(u, min(pushed, edges[id].cap - edges
                [id].flow));
            if (tr == 0) continue;
            edges[id].flow += tr; edges[id ^ 1].flow -= tr;
            return tr;
        }
        return 0;
    }
    int flow() {
        int f = 0;
        while (true) {
            fill(level.begin(), level.end(), -1);
            level[s] = 0;
            q.push(s);
            if (!bfs())
                break;
            fill(ptr.begin(), ptr.end(), 0);
            while (int pushed = dfs(s, flow_inf)) {
                f += pushed;
            }
        }
        return f;
    }
    vector<int> min_cut() {
        vector<int> vis(n, 0);
        queue<int> q;
        q.push(s);
        vis[s] = 1;
        while (!q.empty()) {
            int v = q.front();
            q.pop();
            for (int id : adj[v]) {
                int u = edges[id].u;
```

```

        if (!vis[u] && edges[id].cap > edges[id].flow)
        {
            vis[u] = 1;
            q.push(u);
        }
    }
}
return vis; // vis[v] = 1 S-side of cut
};

```

3.2 edmondkarp

```

// edmonds karp ve~2 call while(bfs(n) != 0)
typedef struct node {
    int vis; int p; vector<int> adj; vector<ll> cap;
} node;
node graph[519];
ll bfs(int n) {
    queue<int> qu;
    vector<bool> vis(510,0);
    vis[1] = 1;
    qu.push(1);
    while(!qu.empty()) {
        int cur = qu.front();
        // printf("cur-- %d\n",cur);
        qu.pop();
        f(i,graph[cur].adj.size()) {
            int nex=graph[cur].adj[i];
            if(vis[nex] == 0 && graph[cur].cap[nex] > 0) {
                graph[nex].p = cur; vis[nex]= 1;
                if(nex==n) break;
                qu.push(nex);
            }
        }
    }
    if(vis[n] == 0) return 0;
    ll maxflow = LLONG_MAX;
    int cur = n;
    while(cur != 1) {
        maxflow = min(maxflow, graph[graph[cur].p].cap[cur]);
        cur = graph[cur].p;
        // printf("%d ",cur);
    }
    // printf("max %d\n", maxflow);
    cur = n;
    while(cur != 1) {
        graph[graph[cur].p].cap[cur] -= maxflow;

```

```

        graph[cur].cap[graph[cur].p] += maxflow;
        cur = graph[cur].p;
    }
    return maxflow;
}

// call after getting max flow
vector<int> getCut(int n) {
    vector<int> vis(510,0);
    queue<int> q;
    q.push(1);
    vis[1] = 1;
    while(!q.empty()) {
        int u = q.front(); q.pop();
        for(int v : graph[u].adj) {
            if(!vis[v] && graph[u].cap[v] > 0) { // residual
                capacity
                vis[v] = 1;
                q.push(v);
            }
        }
    }
    return vis; // vis[v] = 1 -> reachable -> S-side
}

```

3.3 mcmf

```

struct Edge
{
    int from, to, capacity, cost;
};
vector<vector<int>> adj, cost, capacity;
const int INF = 1e9;
void shortest_paths(int n, int v0, vector<int>& d, vector<
    int>& p) {
    d.assign(n, INF);
    d[v0] = 0;
    vector<bool> inq(n, false);
    queue<int> q;
    q.push(v0);
    p.assign(n, -1);
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        inq[u] = false;
        for (int v : adj[u]) {
            if (capacity[u][v] > 0 && d[v] > d[u] + cost[u][v]
                ) {
                d[v] = d[u] + cost[u][v];

```

```

                p[v] = u;
                if (!inq[v]) {
                    inq[v] = true;
                    q.push(v);
                }
            }
        }
    }
}

int min_cost_flow(int N, vector<Edge> edges, int K, int s,
    int t) {
    adj.assign(N, vector<int>());
    cost.assign(N, vector<int>(N, 0));
    capacity.assign(N, vector<int>(N, 0));
    for (Edge e : edges) {
        adj[e.from].push_back(e.to);
        adj[e.to].push_back(e.from);
        cost[e.from][e.to] = e.cost;
        cost[e.to][e.from] = -e.cost;
        capacity[e.from][e.to] = e.capacity;
    }
    int flow = 0;
    int cost = 0;
    vector<int> d, p;
    while (flow < K) {
        shortest_paths(N, s, d, p);
        if (d[t] == INF)
            break;
        // find max flow on that path
        int f = K - flow;
        int cur = t;
        while (cur != s) {
            f = min(f, capacity[p[cur]][cur]);
            cur = p[cur];
        }
        // apply flow
        flow += f;
        cost += f * d[t];
        cur = t;
        while (cur != s) {
            capacity[p[cur]][cur] -= f;
            capacity[cur][p[cur]] += f;
            cur = p[cur];
        }
    }
    if (flow < K)
        return -1;
    else
        return cost;
}

```

4 Geometry

4.1 Convex hull

```
vector<PT> convex_hull(vector<PT> &p) {
    if (p.size() <= 1) return p;
    vector<PT> v = p;
    sort(v.begin(), v.end());
    vector<PT> up, dn;
    for (auto& p : v) {
        while (up.size() > 1 && orientation(up[up.size() - 2], up.back(), p) >= 0) {
            up.pop_back();
        }
        while (dn.size() > 1 && orientation(dn[dn.size() - 2], dn.back(), p) <= 0) {
            dn.pop_back();
        }
        up.push_back(p);
        dn.push_back(p);
    }
    v = dn;
    if (v.size() > 1) v.pop_back();
    reverse(up.begin(), up.end());
    up.pop_back();
    for (auto& p : up) {
        v.push_back(p);
    }
    if (v.size() == 2 && v[0] == v[1]) v.pop_back();
    return v;
}

//checks if convex or not
bool is_convex(vector<PT> &p) {
    bool s[3]; s[0] = s[1] = s[2] = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int k = (j + 1) % n;
        s[sign(cross(p[j] - p[i], p[k] - p[i])) + 1] = 1;
        if (s[0] && s[2]) return 0;
    }
    return 1;
}
```

4.2 Geo 2

```
struct point { int x, y, idx; }; //Finds squared euclidean
distance between two points int dist(point &a, point &b
```

```
) { return (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y); }
//Checks if angle ABC is a right angle int
isOrthogonal(point &a, point &b, point &c) { return (b.
x-a.x) * (b.x-c.x) + (b.y-a.y) * (b.y-c.y) == 0; } //
Checks if ABCD form a rectangle (in that order) int
isRectangle(point &a, point &b, point &c, point &d) {
return isOrthogonal(a, b, c) && isOrthogonal(b, c, d)
&& isOrthogonal(c, d, a); } //Checks if ABCD form a
rectangle, in any orientation int isRectangleAnyOrder(
point &a, point &b, point &c, point &d) { return
isRectangle(a, b, c, d) || isRectangle(b, c, a, d) |
isRectangle(c, a, b, d); } //Checks if ABCD form a
square (in that order) int isSquare(point &a, point &b,
point &c, point &d) { return isRectangle(a, b, c, d)
&& dist(a, b) == dist(b, c); } //Checks if ABCD form a
square, in any orientation int isSquareAnyOrder(point &
a, point &b, point &c, point &d) { return isSquare(a, b
, c, d) || isSquare(b, c, a, d) | isSquare(c, a, b, d);
}
```

4.3 Nearest point

```
const int N = 1<<17; const long long INF =
2000000000000000000; struct vp_node { long long
threshold; pair < int, int > center; vp_node *left, *
right; }; typedef vp_node *node_ptr; int tests,n; pair
< int, int > pts[N],arr[N]; long long ans; node_ptr
root; long long squared_distance(pair < int, int > a,
pair < int, int > b) { return (a.first-b.first)*1ll*(a.
first-b.first) + (a.second-b.second)*1ll*(a.second-b.
second); } struct cmp_points { pair < int, int > a;
cmp_points(){} cmp_points(pair < int, int > p): a(p) {}
bool operator () (const pair < int, int > &x, const
pair < int, int > &y) const { return squared_distance(a
,x)<squared_distance(a,y); } }; void build(node_ptr &
node, int a, int b) { if(a>b) { node=NULL; return; }
node=new vp_node(); if(a==b) { node->threshold=0; node
->center=arr[a]; node->left=NULL; node->right=NULL;
return; } int pos=a+rand()%(b-a+1); swap(arr[a],arr[pos
]); node->center=arr[a]; sort(arr+a+1,arr+b+1,
cmp_points(arr[a])); node->threshold=squared_distance(
arr[a],arr[(a+b)>>1]); build(node->left,a,(a+b)>>1);
build(node->right,((a+b)>>1)+1,b); } void query(
node_ptr &node, pair < int, int > q, long long &ans) {
if(node==NULL) return; if(node->center!=q) ans=min(ans,
squared_distance(node->center,q)); if(node->left==NULL
&& node->right==NULL) { return; } if(squared_distance(q
,node->center)<node->threshold) { query(node->left,q,
ans); if(sqrt(squared_distance(node->center,q))+sqrt(
```

```
ans)>sqrt(node->threshold)) query(node->right,q,ans); }
else { query(node->right,q,ans); if(sqrt(
squared_distance(node->center,q))-sqrt(ans)<sqrt(node->
threshold)) query(node->left,q,ans); } } int main() {
int i; scanf("%d", &tests); while(tests--) { scanf("%d"
, &n); for(i=1;i<=n;i++) { scanf("%d %d", &arr[i].first
, &arr[i].second); pts[i]=arr[i]; } build(root,1,n);
for(i=1;i<=n;i++) { ans=INF; query(root,pts[i],ans);
printf("%lld\n", ans); } } return 0; }
```

4.4 geo2d

```
const double PI = acos((double)-1.0); int sign(double x) {
return (x > eps) - (x < -eps); } struct PT { double x,
y; PT() { x = 0, y = 0; } PT(double x, double y) : x(x)
, y(y) {} PT(const PT &p) : x(p.x), y(p.y) {} PT
operator + (const PT &a) const { return PT(x + a.x, y +
a.y); } PT operator - (const PT &a) const { return PT(
x - a.x, y - a.y); } PT operator * (const double a)
const { return PT(x * a, y * a); } friend PT operator *
(const double &a, const PT &b) { return PT(a * b.x, a
* b.y); } PT operator / (const double a) const { return
PT(x / a, y / a); } bool operator == (PT a) const {
return sign(a.x - x) == 0 && sign(a.y - y) == 0; } bool
operator != (PT a) const { return !(*this == a); }
bool operator < (PT a) const { return sign(a.x - x) ==
0 ? y < a.y : x < a.x; } bool operator > (PT a) const {
return sign(a.x - x) == 0 ? y > a.y : x > a.x; }
double norm() { return sqrt(x * x + y * y); } double
norm2() { return x * x + y * y; } PT perp() { return PT
(-y, x); } double arg() { return atan2(y, x); } PT
truncate(double r) { // returns a vector with norm r
and having same direction double k = norm(); if (!sign(
k)) return *this; r /= k; return PT(x * r, y * r); }
istream &operator >> (istream &in, PT &p) { return in >> p.x
>> p.y; }
ostream &operator << (ostream &out, PT &p) { return out << "
(" << p.x << ", " << p.y << ")"; }
inline double dot(PT a, PT b) { return a.x * b.x + a.y * b.y
; }
inline double dist2(PT a, PT b) { return dot(a - b, a - b);
}
inline double dist(PT a, PT b) { return sqrt(dot(a - b, a -
b)); }
inline double cross(PT a, PT b) { return a.x * b.y - a.y * b
.x; }
inline double cross2(PT a, PT b, PT c) { return cross(b - a,
c - a); }
```

```

inline int orientation(PT a, PT b, PT c) { return sign(cross
    (b - a, c - a)); }
PT perp(PT a) { return PT(-a.y, a.x); }
PT rotateccw90(PT a) { return PT(-a.y, a.x); }
PT rotatecw90(PT a) { return PT(a.y, -a.x); }
PT rotateccw(PT a, double t) { return PT(a.x * cos(t) - a.y
    * sin(t), a.x * sin(t) + a.y * cos(t)); }
PT rotatecw(PT a, double t) { return PT(a.x * cos(t) + a.y *
    sin(t), -a.x * sin(t) + a.y * cos(t)); }
double SQ(double x) { return x * x; }
double rad_to_deg(double r) { return (r * 180.0 / PI); }
double deg_to_rad(double d) { return (d * PI / 180.0); }
double get_angle(PT a, PT b) {
    double costheta = dot(a, b) / a.norm() / b.norm();
    return acos(max((double)-1.0, min((double)1.0, costheta))
        );
}
bool is_point_in_angle(PT b, PT a, PT c, PT p) { // does
    point p lie in angle <bac
    assert(orientation(a, b, c) != 0);
    if (orientation(a, c, b) < 0) swap(b, c);
    return orientation(a, c, p) >= 0 && orientation(a, b, p)
        <= 0;
}
bool half(PT p) {
    return p.y > 0.0 || (p.y == 0.0 && p.x < 0.0);
}
void polar_sort(vector<PT> &v) { // sort points in
    counterclockwise
    sort(v.begin(), v.end(), [](PT a, PT b) {
        return make_tuple(half(a), 0.0, a.norm2()) <
            make_tuple(half(b), cross(a, b), b.norm2());
    });
}
void polar_sort(vector<PT> &v, PT o) { // sort points in
    counterclockwise with respect to point o
    sort(v.begin(), v.end(), [&](PT a, PT b) {
        return make_tuple(half(a - o), 0.0, (a - o).norm2())
            < make_tuple(half(b - o), cross(a - o, b - o), (
                b - o).norm2());
    });
}
struct line {
    PT a, b; // goes through points a and b
    PT v; double c; //line form: direction vec [cross] (x, y)
        = c
    line() {}
    //direction vector v and offset c
    line(PT v, double c) : v(v), c(c) {
        auto p = get_points();

```

```

        a = p.first; b = p.second;
    }
    // equation ax + by + c = 0
    line(double _a, double _b, double _c) : v({_b, -_a}), c(-_c
        ) {
        auto p = get_points();
        a = p.first; b = p.second;
    }
    // goes through points p and q
    line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q) {}
    pair<PT, PT> get_points() { //extract any two points
        from this line
        PT p, q; double a = -v.y, b = v.x; // ax + by = c
        if (sign(a) == 0) {
            p = PT(0, c / b); q = PT(1, c / b);
        }
        else if (sign(b) == 0) {
            p = PT(c / a, 0); q = PT(c / a, 1);
        }
        else {
            p = PT(0, c / b); q = PT(1, (c - a) / b);
        }
        return {p, q};
    }
    // ax + by + c = 0
    array<double, 3> get_abc() {
        double a = -v.y, b = v.x; return {a, b, -c};
    }
    // 1 if on the left, -1 if on the right, 0 if on the line
    int side(PT p) { return sign(cross(v, p) - c); }
    // line that is perpendicular to this and goes through
    point p
    line perpendicular_through(PT p) { return {p, p + perp(v)
        }; }
    // translate the line by vector t i.e. shifting it by
    vector t
    line translate(PT t) { return {v, c + cross(v, t)}; }
    // compare two points by their orthogonal projection on
    this line
    // a projection point comes before another if it comes
    first according to vector v
    bool cmp_by_projection(PT p, PT q) { return dot(v, p) <
        dot(v, q); }
    line shift_left(double d) {
        PT z = v.perp().truncate(d);
        return line(a + z, b + z);
    }
};
// find a point from a through b with distance d
PT point_along_line(PT a, PT b, double d) {

```

```

    assert(a != b);
    return a + (((b - a) / (b - a).norm()) * d);
}
// projection point c onto line through a and b assuming a
    != b
PT project_from_point_to_line(PT a, PT b, PT c) {
    return a + (b - a) * dot(c - a, b - a) / (b - a).norm2();
}
// reflection point c onto line through a and b assuming a
    != b
PT reflection_from_point_to_line(PT a, PT b, PT c) {
    PT p = project_from_point_to_line(a, b, c);
    return p + p - c;
}
// minimum distance from point c to line through a and b
double dist_from_point_to_line(PT a, PT b, PT c) {
    return fabs(cross(b - a, c - a) / (b - a).norm());
}
// returns true if point p is on line segment ab
bool is_point_on_seg(PT a, PT b, PT p) {
    if (fabs(cross(p - b, a - b)) < eps) {
        if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x)
            + eps) return false;
        if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y)
            + eps) return false;
        return true;
    }
    return false;
}
// minimum distance point from point c to segment ab that
    lies on segment ab
PT project_from_point_to_seg(PT a, PT b, PT c) {
    double r = dist2(a, b);
    if (sign(r) == 0) return a;
    r = dot(c - a, b - a) / r;
    if (r < 0) return a;
    if (r > 1) return b;
    return a + (b - a) * r;
}
// minimum distance from point c to segment ab
double dist_from_point_to_seg(PT a, PT b, PT c) {
    return dist(c, project_from_point_to_seg(a, b, c));
}
// 0 if not parallel, 1 if parallel, 2 if collinear
int is_parallel(PT a, PT b, PT c, PT d) {
    double k = fabs(cross(b - a, d - c));
    if (k < eps) {
        if (fabs(cross(a - b, a - c)) < eps && fabs(cross(c -
            d, c - a)) < eps) return 2;
        else return 1;
    }

```

```

    }
    else return 0;
}
// check if two lines are same
bool are_lines_same(PT a, PT b, PT c, PT d) {
    if (fabs(cross(a - c, c - d)) < eps && fabs(cross(b - c,
        c - d)) < eps) return true;
    return false;
}
// bisector vector of <abc
PT angle_bisector(PT &a, PT &b, PT &c){
    PT p = a - b, q = c - b;
    return p + q * sqrt(dot(p, p) / dot(q, q));
}
// 1 if point is ccw to the line, 2 if point is cw to the
// line, 3 if point is on the line
int point_line_relation(PT a, PT b, PT p) {
    int c = sign(cross(p - a, b - a));
    if (c < 0) return 1;
    if (c > 0) return 2;
    return 3;
}
// intersection point between ab and cd assuming unique
// intersection exists
bool line_line_intersection(PT a, PT b, PT c, PT d, PT &ans) {
    double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
    double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d);
    double det = a1 * b2 - a2 * b1;
    if (det == 0) return 0;
    ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2) /
        det);
    return 1;
}
// intersection point between segment ab and segment cd
// assuming unique intersection exists
bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans) {
    double oa = cross2(c, d, a), ob = cross2(c, d, b);
    double oc = cross2(a, b, c), od = cross2(a, b, d);
    if (oa * ob < 0 && oc * od < 0){
        ans = (a * ob - b * oa) / (ob - oa);
        return 1;
    }
    else return 0;
}
// intersection point between segment ab and segment cd
// assuming unique intersection may not exists
// se.size()==0 means no intersection
// se.size()==1 means one intersection
// se.size()==2 means range intersection

```

```

set<PT> seg_seg_intersection_inside(PT a, PT b, PT c, PT d)
{
    PT ans;
    if (seg_seg_intersection(a, b, c, d, ans)) return {ans};
    set<PT> se;
    if (is_point_on_seg(c, d, a)) se.insert(a);
    if (is_point_on_seg(c, d, b)) se.insert(b);
    if (is_point_on_seg(a, b, c)) se.insert(c);
    if (is_point_on_seg(a, b, d)) se.insert(d);
    return se;
}
// intersection between segment ab and line cd
// 0 if do not intersect, 1 if proper intersect, 2 if
// segment intersect
int seg_line_relation(PT a, PT b, PT c, PT d) {
    double p = cross2(c, d, a);
    double q = cross2(c, d, b);
    if (sign(p) == 0 && sign(q) == 0) return 2;
    else if (p * q < 0) return 1;
    else return 0;
}
// intersection between segment ab and line cd assuming
// unique intersection exists
bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &ans) {
    bool k = seg_line_relation(a, b, c, d);
    assert(k != 2);
    if (k) line_line_intersection(a, b, c, d, ans);
    return k;
}
// minimum distance from segment ab to segment cd
double dist_from_seg_to_seg(PT a, PT b, PT c, PT d) {
    PT dummy;
    if (seg_seg_intersection(a, b, c, d, dummy)) return 0.0;
    else return min({dist_from_point_to_seg(a, b, c),
        dist_from_point_to_seg(a, b, d),
        dist_from_point_to_seg(c, d, a),
        dist_from_point_to_seg(c, d, b)});
}
// minimum distance from point c to ray (starting point a
// and direction vector b)
double dist_from_point_to_ray(PT a, PT b, PT c) {
    b = a + b;
    double r = dot(c - a, b - a);
    if (r < 0.0) return dist(c, a);
    return dist_from_point_to_line(a, b, c);
}
// starting point as and direction vector ad
bool ray_ray_intersection(PT as, PT ad, PT bs, PT bd) {
    double dx = bs.x - as.x, dy = bs.y - as.y;

```

```

    double det = bd.x * ad.y - bd.y * ad.x;
    if (fabs(det) < eps) return 0;
    double u = (dy * bd.x - dx * bd.y) / det;
    double v = (dy * ad.x - dx * ad.y) / det;
    if (sign(u) >= 0 && sign(v) >= 0) return 1;
    else return 0;
}
double ray_ray_distance(PT as, PT ad, PT bs, PT bd) {
    if (ray_ray_intersection(as, ad, bs, bd)) return 0.0;
    double ans = dist_from_point_to_ray(as, ad, bs);
    ans = min(ans, dist_from_point_to_ray(bs, bd, as));
    return ans;
}
struct circle {
    PT p; double r;
    circle() {}
    circle(PT _p, double _r): p(_p), r(_r) {};
    // center (x, y) and radius r
    circle(double x, double y, double _r): p(PT(x, y)), r(_r)
        {};
    // circumcircle of a triangle
    // the three points must be unique
    circle(PT a, PT b, PT c) {
        b = (a + b) * 0.5;
        c = (a + c) * 0.5;
        line_line_intersection(b, b + rotatecw90(a - b), c, c
            + rotatecw90(a - c), p);
        r = dist(a, p);
    }
    // inscribed circle of a triangle
    // pass a bool just to differentiate from circumcircle
    circle(PT a, PT b, PT c, bool t) {
        line u, v;
        double m = atan2(b.y - a.y, b.x - a.x), n = atan2(c.y
            - a.y, c.x - a.x);
        u.a = a;
        u.b = u.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
        v.a = b;
        m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y,
            c.x - b.x);
        v.b = v.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
        line_line_intersection(u.a, u.b, v.a, v.b, p);
        r = dist_from_point_to_seg(a, b, p);
    }
    bool operator == (circle v) { return p == v.p && sign(r -
        v.r) == 0; }
    double area() { return PI * r * r; }
    double circumference() { return 2.0 * PI * r; }
};
//0 if outside, 1 if on circumference, 2 if inside circle

```

```

int circle_point_relation(PT p, double r, PT b) {
    double d = dist(p, b);
    if (sign(d - r) < 0) return 2;
    if (sign(d - r) == 0) return 1;
    return 0;
}
// 0 if outside, 1 if on circumference, 2 if inside circle
int circle_line_relation(PT p, double r, PT a, PT b) {
    double d = dist_from_point_to_line(a, b, p);
    if (sign(d - r) < 0) return 2;
    if (sign(d - r) == 0) return 1;
    return 0;
}
//compute intersection of line through points a and b with
//circle centered at c with radius r > 0
vector<PT> circle_line_intersection(PT c, double r, PT a, PT
    b) {
    vector<PT> ret;
    b = b - a; a = a - c;
    double A = dot(b, b), B = dot(a, b);
    double C = dot(a, a) - r * r, D = B * B - A * C;
    if (D < -eps) return ret;
    ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
    if (D > eps) ret.push_back(c + a + b * (-B - sqrt(D)) / A
    );
    return ret;
}
//5 - outside and do not intersect
//4 - intersect outside in one point
//3 - intersect in 2 points
//2 - intersect inside in one point
//1 - inside and do not intersect
int circle_circle_relation(PT a, double r, PT b, double R) {
    double d = dist(a, b);
    if (sign(d - r - R) > 0) return 5;
    if (sign(d - r - R) == 0) return 4;
    double l = fabs(r - R);
    if (sign(d - r - R) < 0 && sign(d - l) > 0) return 3;
    if (sign(d - l) == 0) return 2;
    if (sign(d - l) < 0) return 1;
    assert(0); return -1;
}
vector<PT> circle_circle_intersection(PT a, double r, PT b,
    double R) {
    if (a == b && sign(r - R) == 0) return {PT(1e18, 1e18)};
    vector<PT> ret;
    double d = sqrt(dist2(a, b));
    if (d > r + R || d + min(r, R) < max(r, R)) return ret;
    double x = (d * d - R * R + r * r) / (2 * d);
    double y = sqrt(r * r - x * x);

```

```

    PT v = (b - a) / d;
    ret.push_back(a + v * x + rotateccw90(v) * y);
    if (y > 0) ret.push_back(a + v * x - rotateccw90(v) * y);
    return ret;
}
// returns two circle c1, c2 through points a, b and of
// radius r
// 0 if there is no such circle, 1 if one circle, 2 if two
// circle
int get_circle(PT a, PT b, double r, circle &c1, circle &c2)
{
    vector<PT> v = circle_circle_intersection(a, r, b, r);
    int t = v.size();
    if (!t) return 0;
    c1.p = v[0], c1.r = r;
    if (t == 2) c2.p = v[1], c2.r = r;
    return t;
}
// returns two circle c1, c2 which is tangent to line u,
// goes through
// point q and has radius r1; 0 for no circle, 1 if c1 = c2
// , 2 if c1 != c2
int get_circle(line u, PT q, double r1, circle &c1, circle &
    c2) {
    double d = dist_from_point_to_line(u.a, u.b, q);
    if (sign(d - r1 * 2.0) > 0) return 0;
    if (sign(d) == 0) {
        cout << u.v.x << ' ' << u.v.y << '\n';
        c1.p = q + rotateccw90(u.v).truncate(r1);
        c2.p = q + rotatecw90(u.v).truncate(r1);
        c1.r = c2.r = r1;
        return 2;
    }
    line u1 = line(u.a + rotateccw90(u.v).truncate(r1), u.b +
        rotateccw90(u.v).truncate(r1));
    line u2 = line(u.a + rotatecw90(u.v).truncate(r1), u.b +
        rotatecw90(u.v).truncate(r1));
    circle cc = circle(q, r1);
    PT p1, p2; vector<PT> v;
    v = circle_line_intersection(q, r1, u1.a, u1.b);
    if (!v.size()) v = circle_line_intersection(q, r1, u2.a,
        u2.b);
    v.push_back(v[0]);
    p1 = v[0], p2 = v[1];
    c1 = circle(p1, r1);
    if (p1 == p2) {
        c2 = c1;
        return 1;
    }
    c2 = circle(p2, r1);

```

```

    return 2;
}
// returns area of intersection between two circles
double circle_circle_area(PT a, double r1, PT b, double r2)
{
    double d = (a - b).norm();
    if(r1 + r2 < d + eps) return 0;
    if(r1 + d < r2 + eps) return PI * r1 * r1;
    if(r2 + d < r1 + eps) return PI * r2 * r2;
    double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 *
        r1 * d)),
        theta_2 = acos((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d))
        ;
    return r1 * r1 * (theta_1 - sin(2 * theta_1) / 2.) + r2 *
        r2 * (theta_2 - sin(2 * theta_2) / 2.);
}
// tangent lines from point q to the circle
int tangent_lines_from_point(PT p, double r, PT q, line &u,
    line &v) {
    int x = sign(dist2(p, q) - r * r);
    if (x < 0) return 0; // point in circle
    if (x == 0) { // point on circle
        u = line(q, q + rotateccw90(q - p));
        v = u;
        return 1;
    }
    double d = dist(p, q);
    double l = r * r / d;
    double h = sqrt(r * r - l * l);
    u = line(q, p + ((q - p).truncate(l) + (rotateccw90(q - p
        ).truncate(h))));
    v = line(q, p + ((q - p).truncate(l) + (rotatecw90(q - p)
        .truncate(h))));
    return 2;
}
// returns outer tangents line of two circles
// if inner == 1 it returns inner tangent lines
int tangents_lines_from_circle(PT c1, double r1, PT c2,
    double r2, bool inner, line &u, line &v) {
    if (inner) r2 = -r2;
    PT d = c2 - c1;
    double dr = r1 - r2, d2 = d.norm2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) {
        assert(h2 != 0);
        return 0;
    }
    vector<pair<PT, PT>>out;
    for (int tmp: {-1, 1}) {

```

```

    PT v = (d * dr + rotateccw90(d) * sqrt(h2) * tmp) /
        d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
}
u = line(out[0].first, out[0].second);
if (out.size() == 2) v = line(out[1].first, out[1].second);
return 1 + (h2 > 0);
}

double area_of_triangle(PT a, PT b, PT c) {
    return fabs(cross(b - a, c - a) * 0.5);
}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
// outside
int is_point_in_triangle(PT a, PT b, PT c, PT p) {
    if (sign(cross(b - a, c - a)) < 0) swap(b, c);
    int c1 = sign(cross(b - a, p - a));
    int c2 = sign(cross(c - b, p - b));
    int c3 = sign(cross(a - c, p - c));
    if (c1 < 0 || c2 < 0 || c3 < 0) return 1;
    if (c1 + c2 + c3 != 3) return 0;
    return -1;
}

double perimeter(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans += dist(p[i], p[(i + 1) % n]);
    return ans;
}

double area(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1) % n]);
    return fabs(ans) * 0.5;
}
// centroid of a (possibly non-convex) polygon,
// assuming that the coordinates are listed in a clockwise
// or
// counterclockwise fashion. Note that the centroid is often
// known as
// the "center of gravity" or "center of mass".
PT centroid(vector<PT> &p) {
    int n = p.size(); PT c(0, 0);
    double sum = 0;
    for (int i = 0; i < n; i++) sum += cross(p[i], p[(i + 1) % n]);
    double scale = 3.0 * sum;
    for (int i = 0; i < n; i++) {

```

```

        int j = (i + 1) % n;
        c = c + (p[i] + p[j]) * cross(p[i], p[j]);
    }
    return c / scale;
}
// 0 if cw, 1 if ccw
bool get_direction(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1) % n]);
    if (sign(ans) > 0) return 1;
    return 0;
}
// it returns a point such that the sum of distances
// from that point to all points in p is minimum
// O(n log^2 MX)
PT geometric_median(vector<PT> p) {
    auto tot_dist = [&](PT z) {
        double res = 0;
        for (int i = 0; i < p.size(); i++) res += dist(p[i], z);
        return res;
    };
    auto findY = [&](double x) {
        double y1 = -1e5, yr = 1e5;
        for (int i = 0; i < 60; i++) {
            double ym1 = y1 + (yr - y1) / 3;
            double ym2 = yr - (yr - y1) / 3;
            double d1 = tot_dist(PT(x, ym1));
            double d2 = tot_dist(PT(x, ym2));
            if (d1 < d2) yr = ym2;
            else y1 = ym1;
        }
        return pair<double, double> (y1, tot_dist(PT(x, y1)));
    };
    double x1 = -1e5, xr = 1e5;
    for (int i = 0; i < 60; i++) {
        double xm1 = x1 + (xr - x1) / 3;
        double xm2 = xr - (xr - x1) / 3;
        double y1, d1, y2, d2;
        auto z = findY(xm1); y1 = z.first; d1 = z.second;
        z = findY(xm2); y2 = z.first; d2 = z.second;
        if (d1 < d2) xr = xm2;
        else x1 = xm1;
    }
    return {x1, findY(x1).first};
}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
// outside
// it must be strictly convex, otherwise make it strictly
// convex first

```

```

int is_point_in_convex(vector<PT> &p, const PT& x) { // O(
    log n)
    int n = p.size(); assert(n >= 3);
    int a = orientation(p[0], p[1], x), b = orientation(p[0],
        p[n - 1], x);
    if (a < 0 || b > 0) return 1;
    int l = 1, r = n - 1;
    while (l + 1 < r) {
        int mid = l + r >> 1;
        if (orientation(p[0], p[mid], x) >= 0) l = mid;
        else r = mid;
    }
    int k = orientation(p[l], p[r], x);
    if (k <= 0) return -k;
    if (l == 1 && a == 0) return 0;
    if (r == n - 1 && b == 0) return 0;
    return -1;
}

bool is_point_on_polygon(vector<PT> &p, const PT& z) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        if (is_point_on_seg(p[i], p[(i + 1) % n], z)) return 1;
    }
    return 0;
}
// returns 1e9 if the point is on the polygon
int winding_number(vector<PT> &p, const PT& z) { // O(n)
    if (is_point_on_polygon(p, z)) return 1e9;
    int n = p.size(), ans = 0;
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        bool below = p[i].y < z.y;
        if (below != (p[j].y < z.y)) {
            auto orient = orientation(z, p[j], p[i]);
            if (orient == 0) return 0;
            if (below == (orient > 0)) ans += below ? 1 : -1;
        }
    }
    return ans;
}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
// outside
int is_point_in_polygon(vector<PT> &p, const PT& z) { // O(n)
    int k = winding_number(p, z);
    return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}

double diameter(vector<PT> &p) {
    int n = (int)p.size();

```

```

if (n == 1) return 0;
if (n == 2) return dist(p[0], p[1]);
double ans = 0;
int i = 0, j = 1;
while (i < n) {
    while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] - p[j]) >= 0) {
        ans = max(ans, dist2(p[i], p[j]));
        j = (j + 1) % n;
    }
    ans = max(ans, dist2(p[i], p[j]));
    i++;
}
return sqrt(ans);
}
// minimum distance between two parallel lines (non
// necessarily axis parallel)
// such that the polygon can be put between the lines
double width(vector<PT> &p) {
    int n = (int)p.size();
    if (n <= 2) return 0;
    double ans = inf;
    int i = 0, j = 1;
    while (i < n) {
        while (cross(p[(i + 1) % n] - p[i], p[(j + 1) % n] - p[j]) >= 0) j = (j + 1) % n;
        ans = min(ans, dist_from_point_to_line(p[i], p[(i + 1) % n], p[j]));
        i++;
    }
    return ans;
}
// minimum perimeter
double minimum_enclosing_rectangle(vector<PT> &p) {
    int n = p.size();
    if (n <= 2) return perimeter(p);
    int mndot = 0; double tmp = dot(p[1] - p[0], p[0]);
    for (int i = 1; i < n; i++) {
        if (dot(p[1] - p[0], p[i]) <= tmp) {
            tmp = dot(p[1] - p[0], p[i]);
            mndot = i;
        }
    }
    double ans = inf;
    int i = 0, j = 1, mxdot = 1;
    while (i < n) {
        PT cur = p[(i + 1) % n] - p[i];
        while (cross(cur, p[(j + 1) % n] - p[j]) >= 0) j = (j + 1) % n;
    }

```

```

        while (dot(p[(mxdot + 1) % n], cur) >= dot(p[mxdot], cur)) mxdot = (mxdot + 1) % n;
        while (dot(p[(mndot + 1) % n], cur) <= dot(p[mndot], cur)) mndot = (mndot + 1) % n;
        ans = min(ans, 2.0 * ((dot(p[mxdot], cur) / cur.norm() - dot(p[mndot], cur) / cur.norm()) + dist_from_point_to_line(p[i], p[(i + 1) % n], p[j])));
        i++;
    }
    return ans;
}
// given n points, find the minimum enclosing circle of the points
// call convex_hull() before this for faster solution
// expected O(n)
circle minimum_enclosing_circle(vector<PT> &p) {
    random_shuffle(p.begin(), p.end());
    int n = p.size();
    circle c(p[0], 0);
    for (int i = 1; i < n; i++) {
        if (sign(dist(c.p, p[i]) - c.r) > 0) {
            c = circle(p[i], 0);
            for (int j = 0; j < i; j++) {
                if (sign(dist(c.p, p[j]) - c.r) > 0) {
                    c = circle((p[i] + p[j]) / 2, dist(p[i], p[j]) / 2);
                    for (int k = 0; k < j; k++) {
                        if (sign(dist(c.p, p[k]) - c.r) > 0) {
                            c = circle(p[i], p[j], p[k]);
                        }
                    }
                }
            }
        }
    }
    return c;
}
pair<PT, int> point_poly_tangent(vector<PT> &p, PT Q, int dir, int l, int r) {
    while (r - l > 1) {
        int mid = (l + r) >> 1;
        bool pvs = orientation(Q, p[mid], p[mid - 1]) != -dir;
        bool nxt = orientation(Q, p[mid], p[mid + 1]) != -dir;
        if (pvs && nxt) return {p[mid], mid};
        if (!(pvs || nxt)) {

```

```

            auto p1 = point_poly_tangent(p, Q, dir, mid + 1, r);
            auto p2 = point_poly_tangent(p, Q, dir, l, mid - 1);
            return orientation(Q, p1.first, p2.first) == dir ? p1 : p2;
        }
    }
    if (!pvs) {
        if (orientation(Q, p[mid], p[l]) == dir) r = mid - 1;
        else if (orientation(Q, p[l], p[r]) == dir) r = mid - 1;
        else l = mid + 1;
    }
    if (!nxt) {
        if (orientation(Q, p[mid], p[l]) == dir) l = mid + 1;
        else if (orientation(Q, p[l], p[r]) == dir) r = mid - 1;
        else l = mid + 1;
    }
}
pair<PT, int> ret = {p[l], l};
for (int i = l + 1; i <= r; i++) ret = orientation(Q, ret.first, p[i]) != dir ? make_pair(p[i], i) : ret;
return ret;
}
// (ccw, cw) tangents from a point that is outside this convex polygon
// returns indexes of the points
// ccw means the tangent from Q to that point is in the same direction as the polygon ccw direction
pair<int, int> tangents_from_point_to_polygon(vector<PT> &p, PT Q) {
    int ccw = point_poly_tangent(p, Q, 1, 0, (int)p.size() - 1).second;
    int cw = point_poly_tangent(p, Q, -1, 0, (int)p.size() - 1).second;
    return make_pair(ccw, cw);
}
// minimum distance from a point to a convex polygon
// it assumes point lie strictly outside the polygon
double dist_from_point_to_polygon(vector<PT> &p, PT z) {
    double ans = inf;
    int n = p.size();
    if (n <= 3) {
        for (int i = 0; i < n; i++) ans = min(ans, dist_from_point_to_seg(p[i], p[(i + 1) % n], z));
    }

```

```

    return ans;
}
auto [r, l] = tangents_from_point_to_polygon(p, z);
if(l > r) r += n;
while (l < r) {
    int mid = (l + r) >> 1;
    double left = dist2(p[mid % n], z), right = dist2(p[(mid + 1) % n], z);
    ans = min({ans, left, right});
    if(left < right) r = mid;
    else l = mid + 1;
}
ans = sqrt(ans);
ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l + 1) % n], z));
ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l - 1 + n) % n], z));
return ans;
}
// minimum distance from convex polygon p to line ab
// returns 0 is it intersects with the polygon
// top - upper right vertex
double dist_from_polygon_to_line(vector<PT> &p, PT a, PT b,
    int top) { //O(log n)
    PT orth = (b - a).perp();
    if (orientation(a, b, p[0]) > 0) orth = (a - b).perp();
    int id = extreme_vertex(p, orth, top);
    if (dot(p[id] - a, orth) > 0) return 0.0; //if orth and a
    // are in the same half of the line, then poly and line
    // intersects
    return dist_from_point_to_line(a, b, p[id]); //does not
    // intersect
}
// minimum distance from a convex polygon to another convex
// polygon
// the polygon doesnot overlap or touch
// tested in https://toph.co/p/the-wall
double dist_from_polygon_to_polygon(vector<PT> &p1, vector<
    PT> &p2) { // O(n log n)
    double ans = inf;
    for (int i = 0; i < p1.size(); i++) {
        ans = min(ans, dist_from_point_to_polygon(p2, p1[i]));
    }
    for (int i = 0; i < p2.size(); i++) {
        ans = min(ans, dist_from_point_to_polygon(p1, p2[i]));
    }
    return ans;
}

```

```

// maximum distance from a convex polygon to another convex
// polygon
double maximum_dist_from_polygon_to_polygon(vector<PT> &u,
    vector<PT> &v) { //O(n)
    int n = (int)u.size(), m = (int)v.size();
    double ans = 0;
    if (n < 3 || m < 3) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) ans = max(ans, dist2(
                u[i], v[j]));
        }
        return sqrt(ans);
    }
    if (u[0].x > v[0].x) swap(n, m), swap(u, v);
    int i = 0, j = 0, step = n + m + 10;
    while (j + 1 < m && v[j].x < v[j + 1].x) j++;
    while (step-- > 0) {
        if (cross(u[(i + 1) % n] - u[i], v[(j + 1) % m] - v[j])
            >= 0) j = (j + 1) % m;
        else i = (i + 1) % n;
        ans = max(ans, dist2(u[i], v[j]));
    }
    return sqrt(ans);
}
// rotate the polygon such that the (bottom, left)-most
// point is at the first position
void reorder_polygon(vector<PT> &p) {
    int pos = 0;
    for (int i = 1; i < p.size(); i++) {
        if (p[i].y < p[pos].y || (sign(p[i].y - p[pos].y) == 0 &&
            p[i].x < p[pos].x)) pos = i;
    }
    rotate(p.begin(), p.begin() + pos, p.end());
}
// returns the area of the intersection of the circle with
// center c and radius r
// and the triangle formed by the points c, a, b
double _triangle_circle_intersection(PT c, double r, PT a,
    PT b) {
    double sd1 = dist2(c, a), sd2 = dist2(c, b);
    if(sd1 > sd2) swap(a, b), swap(sd1, sd2);
    double sd = dist2(a, b);
    double d1 = sqrt1(sd1), d2 = sqrt1(sd2), d = sqrt(sd);
    double x = abs(sd2 - sd - sd1) / (2 * d);
    double h = sqrt1(sd1 - x * x);
    if(r >= d2) return h * d / 2;
    double area = 0;
    if(sd + sd1 < sd2) {

```

```

        if(r < d1) area = r * r * (acos(h / d2) - acos(h / d1
            )) / 2;
        else {
            area = r * r * (acos(h / d2) - acos(h / r)) / 2;
            double y = sqrt1(r * r - h * h);
            area += h * (y - x) / 2;
        }
    }
    else {
        if(r < h) area = r * r * (acos(h / d2) + acos(h / d1
            )) / 2;
        else {
            area += r * r * (acos(h / d2) - acos(h / r)) / 2;
            double y = sqrt1(r * r - h * h);
            area += h * y / 2;
            if(r < d1) {
                area += r * r * (acos(h / d1) - acos(h / r)) /
                    2;
                area += h * y / 2;
            }
            else area += h * x / 2;
        }
    }
    return area;
}
// intersection between a simple polygon and a circle
double polygon_circle_intersection(vector<PT> &v, PT p,
    double r) {
    int n = v.size();
    double ans = 0.00;
    PT org = {0, 0};
    for(int i = 0; i < n; i++) {
        int x = orientation(p, v[i], v[(i + 1) % n]);
        if(x == 0) continue;
        double area = _triangle_circle_intersection(org, r, v
            [i] - p, v[(i + 1) % n] - p);
        if (x < 0) ans -= area;
        else ans += area;
    }
    return abs(ans);
}
// find a circle of radius r that contains as many points as
// possible
// O(n^2 log n);
double maximum_circle_cover(vector<PT> p, double r, circle &
    c) {
    int n = p.size();
    int ans = 0;
    int id = 0; double th = 0;
    for (int i = 0; i < n; ++i) {

```

```

// maximum circle cover when the circle goes through
// this point
vector<pair<double, int>> events = {{-PI, +1}, {PI,
-1}};
for (int j = 0; j < n; ++j) {
    if (j == i) continue;
    double d = dist(p[i], p[j]);
    if (d > r * 2) continue;
    double dir = (p[j] - p[i]).arg();
    double ang = acos(d / 2 / r);
    double st = dir - ang, ed = dir + ang;
    if (st > PI) st -= PI * 2;
    if (st <= -PI) st += PI * 2;
    if (ed > PI) ed -= PI * 2;
    if (ed <= -PI) ed += PI * 2;
    events.push_back({st - eps, +1}); // take care of
    // precisions!
    events.push_back({ed, -1});
    if (st > ed) {
        events.push_back({-PI, +1});
        events.push_back({+PI, -1});
    }
}
sort(events.begin(), events.end());
int cnt = 0;
for (auto &&e: events) {
    cnt += e.second;
    if (cnt > ans) {
        ans = cnt;
        id = i; th = e.first;
    }
}
}
PT w = PT(p[id].x + r * cos(th), p[id].y + r * sin(th));
c = circle(w, r); //best_circle
return ans;
}
// radius of the maximum inscribed circle in a convex
// polygon
double maximum_inscribed_circle(vector<PT> p) {
    int n = p.size();
    if (n <= 2) return 0;
    double l = 0, r = 20000;
    while (r - l > eps) {
        double mid = (l + r) * 0.5;
        vector<HP> h;
        const int L = 1e9;
        h.push_back(HP(PT(-L, -L), PT(L, -L)));
        h.push_back(HP(PT(L, -L), PT(L, L)));
        h.push_back(HP(PT(L, L), PT(-L, L)));

```

```

        h.push_back(HP(PT(-L, L), PT(-L, -L)));
        for (int i = 0; i < n; i++) {
            PT z = (p[(i + 1) % n] - p[i]).perp();
            z = z.truncate(mid);
            PT y = p[i] + z, q = p[(i + 1) % n] + z;
            h.push_back(HP(p[i] + z, p[(i + 1) % n] + z));
        }
        vector<PT> nw = half_plane_intersection(h);
        if (!nw.empty()) l = mid;
        else r = mid;
    }
    return l;
}

```

5 Graph

5.1 Articulation Point_{Bridge}

```

int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children=0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] >= tin[v] && p != -1)
                IS_CUTPOINT(v);
            ++children;
        }
    }
    if (p == -1 && children > 1)
        IS_CUTPOINT(v);
}
void find_cutpoints() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {

```

```

        if (!visited[i])
            dfs(i);
    }
}
// finding bridge.
void IS_BRIDGE(int v, int to); // some function to process
// the found bridge
int n; // number of nodes
vector<vector<int>> adj; // adjacency list of graph
vector<bool> visited;
vector<int> tin, low;
int timer;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    bool parent_skipped = false;
    for (int to : adj[v]) {
        if (to == p && !parent_skipped) {
            parent_skipped = true;
            continue;
        }
        if (visited[to]) {
            low[v] = min(low[v], tin[to]);
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v])
                IS_BRIDGE(v, to);
        }
    }
}
void find_bridges() {
    timer = 0;
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    for (int i = 0; i < n; ++i) {
        if (!visited[i])
            dfs(i);
    }
}

```

5.2 Bell

```

void solve()
{
    vector<int> d(n, INF);
    d[v] = 0;
    vector<int> p(n, -1);

```

```

int x;
for (int i = 0; i < n; ++i) {
    x = -1;
    for (Edge e : edges)
        if (d[e.a] < INF)
            if (d[e.b] > d[e.a] + e.cost) {
                d[e.b] = max(-INF, d[e.a] + e.cost);
                p[e.b] = e.a;
                x = e.b;
            }
}
if (x == -1)
    cout << "No negative cycle from " << v;
else {
    int y = x;
    for (int i = 0; i < n; ++i)
        y = p[y];
    vector<int> path;
    for (int cur = y; cur = p[cur]) {
        path.push_back(cur);
        if (cur == y && path.size() > 1)
            break;
    }
    reverse(path.begin(), path.end());
    cout << "Negative cycle: ";
    for (int u : path)
        cout << u << ' ';
}
}

```

5.3 euler_{path directed}

```

vector<int> adj[200005];
int ptrs[200005];
int in_deg[200005], out_deg[200005];
vector<int> path; // node sequence
vector<int> edges; // edge sequence
int to[200005];
bool used[200005];

void dfs(int u) {
    while(ptrs[u] < adj[u].size()) {
        int e = adj[u][ptrs[u]++];
        if(used[e]) continue;
        used[e] = true;
        dfs(to[e]);
        edges.push_back(e);
    }
    path.push_back(u);
}

```

```

}

for(int i=0;i<m;i++){
    int u,v;
    cin >> u >> v;
    adj[u].push_back(i);
    to[i] = v;
    out_deg[u]++;
    in_deg[v]++;
    if(i==0) s = u;
}

// degree check for Euler path
int start = -1, end = -1;
bool ok = true;

for(int i=0;i<n;i++){
    int diff = out_deg[i] - in_deg[i];
    if(diff == 1){
        if(start != -1) ok = false;
        start = i;
    }
    else if(diff == -1){
        if(end != -1) ok = false;
        end = i;
    }
    else if(diff != 0){
        ok = false;
    }
}

reverse(path.begin(), path.end());
reverse(edges.begin(), edges.end());
}

```

5.4 kosaraju

```

typedef struct node {
    int vis;
    vector<int> adj;
    vector<int> ulta;
} node;
node tree[300010];
vector<vector<int>> scc(300010);
stack<int> st;
int scount = 0;
void dfs(int u) {
    tree[u].vis = 1;
}

```

```

f(i,tree[u].adj.size()) {
    int next = tree[u].adj[i];
    if(tree[next].vis==0) dfs(next);
}
st.push(u);
}

void dfs2(int u) {
    // printf("%d %d\n",u,scount);
    tree[u].vis = 1;
    scc[scount-1].push_back(u);
    f(i,tree[u].ulta.size()) {
        int next = tree[u].ulta[i];
        if(tree[next].vis==0) dfs2(next);
    }
}

```

5.5 mos on trees

```

struct Query { ll id, l, r, bno,k; }; //basic comparator
bool cmp(Query &a,Query &b) { if (a.bno != b.bno) {
    return a.bno < b.bno; } else if (a.bno & 1) { return a.
    r < b.r; } else { return a.r > b.r; } } ll n,q; ll bs
    =700; // change is depending on problem Query qry[N]; ll
    a[N],b[N],st[N],en[N]; ll ans[N]; ll tt = -1; vll g[N
    ]; void gg(ll x, ll pr){ //trc(x); tt++; st[x] = tt; a[
    tt] = x; //trc(tt,a[tt]); go(i,g[x]){ if(i == pr) cu;
    gg(i,x); } tt++; en[x] = tt; a[tt] = x; } const ll LG =
    30; ll tim=0; ll parent[LG][N]; ll tin[N], tout[N],
    level[N]; void dfs(ll k, ll par, ll lvl) { tin[k]=++tim
    ; parent[0][k]=par; level[k]=lvl; for(auto it:g[k]) {
    if(it==par) continue; dfs(it, k, lvl+1); } tout[k]=tim;
    } ll walk(ll u, ll h) { for(ll i=LG-1;i>=0;i--) { if((
    h>>i) & 1) u = parent[i][u]; } return u; } void
    precompute() { for(ll i=1;i<LG;i++) for(ll j=1;j<=n;j
    ++){ if(parent[i-1][j]) parent[i][j]=parent[i-1][parent
    [i-1][j]]; } ll lca(ll u, ll v) { if(level[u]<level[v])
    swap(u,v); ll diff=level[u]-level[v]; for(ll i=LG-1;i
    >=0;i--) { if((1<<i) & diff) { u=parent[i][u]; } } if(u
    ==v) return u; for(ll i=LG-1;i>=0;i--) { if(parent[i][u
    ] && parent[i][u]!=parent[i][v]) { u=parent[i][u]; v=
    parent[i][v]; } } return parent[0][u]; } ll dist(ll u,
    ll v) { return level[u] + level[v] - 2 * level[lca(u, v
    )]; } ll lp,rp,cnt; ll freq[1000001]; ll freq2
    [1000005]; //unordered_map<ll,ll>freq,freq2; void add(
    ll val) { freq[val]++; freq2[b[val]]++; if(freq[val] ==
    2){ freq2[b[val]]-=2; // if(freq2[b[val]] == 0){ //
    cnt--; // } } else{ // if(freq2[b[val]] == 1) cnt++; }
    } void remove(ll val) { freq[val]--; freq2[b[val]]--;
    if(freq[val] == 1){ freq2[b[val]]+=2; // if(freq2[b[val]
    }
}

```

```

]] == 1){ // cnt++; // } } else{ // if(freq2[b[val]] ==
0) cnt--; } } ll get_answer() { return cnt; } void
solve() { cin >> n>>q; //trc(n,q); mll m; ll dd = 0;
for (ll i = 1; i <= n; i++) { cin >> b[i]; if(!m.count(
b[i])){ m[b[i]] = dd++; } b[i] = m[b[i]]; } f0(n-1){
ipp(x,y); //trc(x,y); g[x].pb(y); g[y].pb(x); } gg(1,0)
; dfs(1,0,0); precompute(); for (ll i = 0; i < q; i++)
{ ll x, y; cin >> x >> y; if(level[x]>level[y]){ swap(x
,y); } ll p = lca(x,y); // trc(x,y,p); ll l,r; if(p ==
x){ l = st[x]; r = st[y]; qry[i].k = -1; } else{ if(st[
x]>st[y]) swap(x,y); l = en[x]; r = st[y]; qry[i].k =
st[p]; // trc(l,r,p,st[p]); // f(j,l,r+1){ // trc(j,a[j
]); // } } qry[i].id = i; qry[i].l = l; qry[i].r = r;
qry[i].bno = l / bs; } // sort the queries sort(qry,
qry + q, cmp); // initialise the data-structure lp = 0;
rp = -1; cnt = 0; // add(a[0]); // answer the queries
for (ll i = 0; i < q; i++) { ll l = qry[i].l; ll r =
qry[i].r; while (rp < r) { add(a[+rp]); } while (lp >
l) { add(a[--lp]); } while (rp > r) { remove(a[rp--]);
} while (lp < l) { remove(a[lp++]); } ans[qry[i].id] =
get_answer(); if(qry[i].k != -1){ // if(freq2[b[a[qry[i
].k]]] == 0){ // ans[qry[i].id]++; // } } } for (ll i =
0; i < q; i++) { cout << ans[i] << endl; } } // this
require the element to be added on both side l,r and
removed l,r // but we can also use mo's when data
structure allow the removal from the side where we
added like rollback dsu void fun() { solve(); } int32_t
main() { // tasklist // taskkill /f /im a.exe
depressed(); ll tc1 = 1; //cin>>tc1; while(tc1--) { fun
(); } //cer; return 0; }

```

5.6 roll_{dsu}

```

struct DSU { int connected; int sz[N]; pair<int, int> par[N
]; //stores <parent, parity of distance from parent>
vector<int> changes; void init(int n) { for(int i=1;i<=
n;i++) { par[i]={i, 0}; sz[i]=1; } connected=n; } pair<
int, int> getPar(int x) { if(par[x].first==x) return
par[x]; pair<int, int> p = getPar(par[x].first); p.
second ^= par[x].second; return p; } bool unite(int u,
int v) //Returns 1 if the graph is no longer bipartite
{ pair<int, int> x=getPar(u), y=getPar(v); if(x.first==
y.first) { if(x.second==y.second) return 1; changes.
push_back(-1); return 0; } connected--; if(sz[x.first]
< sz[y.first]) swap(x, y); par[y.first]={x.first, 1^x.
second^y.second}; sz[x.first]+=sz[y.first]; changes.
push_back(y.first); return 0; } void undo() //Reversed
the previous performed operation { if(!changes.size())
return; if(changes.back()!=-1) { int x=changes.back();

```

```

sz[par[x].first]-=sz[x]; par[x]={x, 0}; } changes.
pop_back(); } };
```

6 Number Theory Math

6.1 CRT

```

__int128 ExU(__int128 a, __int128 b, __int128 &x, __int128 &
y) {
if(b == 0) {
x = 1;
y = 0;
return a;
}
__int128 x1, y1;
__int128 d = ExU(b, a%b, x1, y1);
x = y1;
y = x1 - (a/b)*y1;
return d;
}
void solve() {
cout << "Case " << ii++ << ": ";
int n; cin >> n;
vector<int> a(n), m(n);
for (int i = 0; i < n; i++) cin >> m[i] >> a[i];
int a1 = a[0], m1 = m[0];
for (int i = 1; i < n; i++) {
__int128 a2 = a[i], m2 = m[i];
__int128 x, y;
__int128 g = ExU(m1, m2, x, y);
if((a1-a2) % g != 0) {
cout << "Impossible" << endl;
return;
}
__int128 k = (a1-a2) / g, lcm = (m1/g) * m2;
__int128 a = a1 - m1*(k * x);
a1 = (a % lcm);
if(a1 < 0) a1 += lcm;
m1 = lcm;
}
cout << a1 << endl;
}

```

6.2 Diophantile

```

int gcd(int a, int b, int& x, int& y) {

```

```

if (b == 0) { x = 1; y = 0; return a;
}
int x1, y1;
int d = gcd(b, a % b, x1, y1);
x = y1; y = x1 - y1 * (a / b);
return d;
}
bool find_any_solution(int a, int b, int c, int &x0, int &y0
, int &g) {
g = gcd(abs(a), abs(b), x0, y0);
if (c % g) {
return false;
}
x0 *= c / g; y0 *= c / g;
if (a < 0) x0 = -x0;
if (b < 0) y0 = -y0;
return true;
}
void shift_solution(int &x, int &y, int a, int b, int cnt)
{
x += cnt * b; y -= cnt * a;
}
int find_all_solutions(int a, int b, int c, int minx, int
maxx, int miny, int maxy) {
int x, y, g;
if (!find_any_solution(a, b, c, x, y, g)) return 0;
a /= g; b /= g;
int sign_a = a > 0 ? +1 : -1;
int sign_b = b > 0 ? +1 : -1;
shift_solution(x, y, a, b, (minx - x) / b);
if (x < minx) shift_solution(x, y, a, b, sign_b);
if (x > maxx) return 0;
int lx1 = x;
shift_solution(x, y, a, b, (maxx - x) / b);
if (x > maxx) shift_solution(x, y, a, b, -sign_b);
int rx1 = x; shift_solution(x, y, a, b, -(miny - y) / a);
if (y < miny) shift_solution(x, y, a, b, -sign_a);
if (y > maxy) return 0;
int lx2 = x;
shift_solution(x, y, a, b, -(maxy - y) / a);
if (y > maxy) shift_solution(x, y, a, b, sign_a);
int rx2 = x;
if (lx2 > rx2) swap(lx2, rx2);
int lx = max(lx1, lx2);
int rx = min(rx1, rx2);
if (lx > rx) return 0;
return (rx - lx) / abs(b) + 1;
}

```

6.3 FFT

```
const int N = 3e5 + 9;
const double PI = acos(-1);
struct base {
    double a, b;
    base(double a = 0, double b = 0) : a(a), b(b) {}
    const base operator + (const base &c) const
    { return base(a + c.a, b + c.b); }
    const base operator - (const base &c) const
    { return base(a - c.a, b - c.b); }
    const base operator * (const base &c) const
    { return base(a * c.a - b * c.b, a * c.b + b * c.a); }
};
void fft(vector<base> &p, bool inv = 0) {
    int n = p.size(), i = 0;
    for(int j = 1; j < n - 1; ++j) {
        for(int k = n >> 1; k > (i ^ k); k >>= 1);
        if(j < i) swap(p[i], p[j]);
    }
    for(int l = 1, m; (m = l << 1) <= n; l <= m) {
        double ang = 2 * PI / m;
        base wn = base(cos(ang), (inv ? 1. : -1.) * sin(ang));
        for(int i = 0, j, k; i < n; i += m) {
            for(w = base(1, 0), j = i, k = i + 1; j < k; ++j, w = w * wn) {
                base t = w * p[j + 1];
                p[j + 1] = p[j] - t;
                p[j] = p[j] + t;
            }
        }
    }
    if(inv) for(int i = 0; i < n; ++i) p[i].a /= n, p[i].b /= n;
}
vector<long long> multiply(vector<int> &a, vector<int> &b) {
    int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
    while(sz < t) sz <= 1;
    vector<base> x(sz), y(sz);
    for(int i = 0; i < sz; ++i) {
        x[i] = i < (int)a.size() ? base(a[i], 0) : base(0, 0);
        y[i] = i < (int)b.size() ? base(b[i], 0) : base(0, 0);
    }
    fft(x), fft(y);
    for(int i = 0; i < sz; ++i) z[i] = x[i] * y[i];
    fft(z, 1);
    vector<long long> ret(sz);
    for(int i = 0; i < sz; ++i) ret[i] = (long long) round(z[i].a);
}
```

```
while((int)ret.size() > 1 && ret.back() == 0) ret.pop_back();
return ret;
}
```

6.4 LSieveBigModTernaryS

```
int a[], n elements;
int mx = 0; int ans = 0;
for (int i = 0; i < n; i++) {
    mx = max(mx+a[i], (int)0);
    ans = max(ans, mx);
}
#define sz 1000
vector<int> prime;
bitset<sz> bs;
void sieve() {
    bs.set(); // all bit = 1..
    bs[0] = bs[1] = 0;
    prime.push_back(2);
    for (int i = 3; i <= sz; i += 2) {
        if(bs[i]) {
            for (int j = i*i; j <= sz; j += i)
                bs[j] = 0;
            prime.push_back(i);
        }
    }
}
void solve()
{
    int n = 12;
    int a[] = {60, 70, 80, 90, 100, 200, 400, 500, 600, 700, 900, 1000};
    int r = n-1; int l = 0;
    // cout << (r-l) / 2;
    while(l < r) {
        int c = (r-l) / 3;
        int m1 = l + c;
        int m2 = r - c;
        // cout << a[m1] << ' ' << a[m2] << endl;
        if(a[m1] < a[m2]) {
            r = m2 - 1;
        } else if(a[m1] > a[m2]) {
            l = m1 + 1;
        } else {
            l = m1 + 1; r = m2 - 1;
        }
    }
    cout << a[l] << endl;
}
```

```
}
int mod = 1e9 + 7;
// linear sieve.
vector<int> prime;
bool is_composite[MAXN];
void sieve (int n) {
    std::fill (is_composite, is_composite + n, false);
    for (int i = 2; i < n; ++i) {
        if (!is_composite[i]) prime.push_back (i);
        for (int j = 0; j < prime.size () && i * prime[j] < n; ++j) {
            is_composite[i * prime[j]] = true;
            if (i % prime[j] == 0) break;
        }
    }
}
```

6.5 Miller Rabin

```
using u64 = uint64_t;
using u128 = __uint128_t;
u64 binpower(u64 base, u64 e, u64 mod) {
    u64 result = 1;
    base %= mod;
    while (e) {
        if (e & 1)
            result = (u128)result * base % mod;
        base = (u128)base * base % mod;
        e >>= 1;
    }
    return result;
}
bool check_composite(u64 n, u64 a, u64 d, int s) {
    u64 x = binpower(a, d, n);
    if (x == 1 || x == n - 1)
        return false;
    for (int r = 1; r < s; r++) {
        x = (u128)x * x % n;
        if (x == n - 1)
            return false;
    }
    return true;
};
bool MillerRabin(u64 n, int iter=5) { // returns true if n
    is probably prime, else returns false.
    if (n < 4)
        return n == 2 || n == 3;
    int s = 0;
    u64 d = n - 1;
```

```

while ((d & 1) == 0) {
    d >>= 1;
    s++;
}
for (int i = 0; i < iter; i++) {
    int a = 2 + rand() % (n - 3);
    if (check_composite(n, a, d, s))
        return false;
}
return true;
}
bool MillerRabin(u64 n) { // returns true if n is prime,
    else returns false.
    if (n < 2)
        return false;
    int r = 0;
    u64 d = n - 1;
    while ((d & 1) == 0) {
        d >>= 1;
        r++;
    }
    for (int a : {2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31,
        37}) {
        if (n == a)
            return true;
        if (check_composite(n, a, d, r))
            return false;
    }
    return true;
}

```

6.6 matexpo

```

vector<vector<ll>> matmultiply(vector<vector<ll>>& a, vector
<vector<ll>>& b) {
    if(a[0].size() != b.size()) return vector<vector<ll>>(0);
    else {
        vector<vector<ll>> ans(a.size(), vector<ll>(b[0].size
(),0));
        f(i,a.size()) {
            f(j,b[0].size()) {
                f(k,b.size()) ans[i][j] = (ans[i][j] + ((a[i][k
]*b[k][j])%mod))%mod;
            }
        }
        return ans;
    }
}
vector<vector<ll>> matexpo(vector<vector<ll>>& a, ll exp) {

```

```

vector<vector<ll>> res(a[0].size(), vector<ll>(a[0].size
(),0));
f(i,a[0].size()) res[i][i] = 1;
while(exp > 0) {
    if(exp&1) res = matmultiply(res,a);
    a = matmultiply(a,a);
    exp /= 2;
}
return res;
}
int main() {
    ll n;
    scanf("%lld", &n);
    if(n==0) printf("0\n");
    else {
        vector<vector<ll>> arr(2,vector<ll>(2,1));
        arr[1][1] = 0;
        arr = matexpo(arr,n);
        printf("%lld\n",arr[0][1]);
    }

    return 0;
}

```

6.7 mobius

```

int mob[N];
void mobius() {
    mob[1] = 1;
    for (int i = 2; i < N; i++){
        mob[i]--;
        for (int j = i + i; j < N; j += i) {
            mob[j] -= mob[i];
        }
    }
}
bool vis[N];
vector<int> d[N];
int mul[N];
void add(int x, int k) {
    for (auto y: d[x]) {
        mul[y] += k;
    }
}
int query(int x) {
    int ans = 0;
    for (auto y: d[x]) {
        ans += mul[y] * mob[y];
    }
}

```

```

return ans;
}

```

6.8 ncr

```

// nCr: (n,0)=1,(n,1)=n,(n,r)=(n-1,r)+(n-1,r-1)
// Circular permutation: nPr/r
ll fact[MAX], inv[MAX], invFact[MAX];
// nCr_mod_p_by_lucas_for_small_p._0(LogN)
ll Lucas(ll n, ll m){
    if (n == 0 && m == 0) return 1ll;
    ll ni = n % MOD;
    ll mi = m % MOD;
    if (mi > ni) return 0;
    return (Lucas(n / MOD, m / MOD) * nCr(ni, mi)) % MOD;
}
ll nCr_by_lucas(ll n, ll r) { return Lucas(n, r); }
// nCr_mod_M, M_is_not_prime
const int N = 142858; // need primes <=M (nCr%M)
int spf[N];
vector<int> primes;
void sieve(){
    for (int i = 2; i < N; i++) {
        if (spf[i] == 0) spf[i] = i, primes.push_back(i);
        int sz = primes.size();
        for (int j = 0; j < sz && i * primes[j] < N && primes
[j] <= spf[i]; j++) spf[i * primes[j]] = primes[
j];
    }
}
// returns n! % mod, mod = multiple of p
// 0(mod * log(n))
int factmod(ll n, int p, const int mod) {
    vector<int> f(mod + 1);
    f[0] = 1 % mod;
    for (int i = 1; i <= mod; i++) {
        if (i % p) f[i] = 1LL * f[i - 1] * i % mod;
        else f[i] = f[i - 1];
    }
    int ans = 1 % mod;
    while (n > 1) {
        ans = 1LL * ans * f[n % mod] % mod;
        ans = 1LL * ans * BigMod(f[mod], n / mod, mod) % mod;
        n /= p;
    }
    return ans;
}
ll multiplicity(ll n, int p) {
    ll ans = 0;

```

```

while (n) {
    n /= p;
    ans += n;
}
return ans;
}
// C(n, r) modulo p^k
// 0(p^k log n)
int ncr(ll n, ll r, int p, int k){
    if (n < r || r < 0) return 0;
    int mod = 1;
    for (int i = 0; i < k; i++) { mod *= p; }
    ll t = multiplicity(n, p) - multiplicity(r, p) -
        multiplicity(n - r, p);
    if (t >= k) return 0;
    int ans = 1LL * factmod(n, p, mod) * inverse(factmod(r, p,
        mod), mod) % mod * inverse(factmod(n - r, p, mod),
        mod) % mod;
    ans = 1LL * ans * BigMod(p, t, mod) % mod;
    return ans;
}
pair<ll, ll> CRT(ll a1, ll m1, ll a2, ll m2){
    ll p, q; ll g = e_gcd(m1, m2, p, q);
    if (a1 % g != a2 % g) return make_pair(0, -1);
    ll m = m1 / g * m2;
    p = (p % m + m) % m;
    q = (q % m + m) % m;
    return make_pair((p * a2 % m * (m1 / g) % m + q * a1 % m
        * (m2 / g) % m) % m, m);
}
// 0(m log(n) log(m))
int ncr(ll n, ll r, int m){
    if (n < r || r < 0) return 0;
    pair<ll, ll> ans({0, 1});
    while (m > 1){
        int p = spf[m], k = 0, cur = 1;
        while (m % p == 0){
            m /= p; cur *= p; ++k;
        }
        ans = CRT(ans.first, ans.second, ncr(n, r, p, k), cur
            );
    }
    return ans.first;
}

```

6.9 stirling_number

```

// S(n,k) = (1/k!) * sum_{i=0..k} (-1)^i * C(k,i) * (k-i)^n
int main() {

```

```

int n, k; cin >> n >> k;
vector<ll> fact(k+1, 1), invfact(k+1, 1);
ll ans = 0;
for (int i = 0; i <= k; i++) {
    ll term = fact[k] * invfact[i] % MOD * invfact[k-i] %
        MOD;
    term = term * power(k - i, n) % MOD;
    if (i & 1) ans = (ans - term + MOD) % MOD;
    else ans = (ans + term) % MOD;
}
ans = ans * modinv(fact[k]) % MOD;
cout << ans << "\n";
}

```

6.10 totient

```

int phi(int n) {
    int result = n;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            while (n % i == 0) n /= i;
            result -= result / i;
        }
    }
    if (n > 1) result -= result / n;
    return result;
}
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    for (int i = 0; i <= n; i++) phi[i] = i;
    for (int i = 2; i <= n; i++) {
        if (phi[i] == i) {
            for (int j = i; j <= n; j += i) phi[j] -= phi[j]
                / i;
        }
    }
}
void phi_1_to_n(int n) {
    vector<int> phi(n + 1);
    phi[0] = 0; phi[1] = 1;
    for (int i = 2; i <= n; i++) phi[i] = i - 1;

    for (int i = 2; i <= n; i++)
        for (int j = 2 * i; j <= n; j += i) phi[j] -= phi[i];
}

```

7 String

7.1 AhoCorasick

```

typedef struct AC {
    static const int K = 26;
    struct node {
        int nxt[K], link, leaf, par;
        vector<int> ind;
        char pch;
        node(int par = -1, char pch = '$') : par(par), pch(
            pch) {
            fill(begin(nxt), end(nxt), -1);
            leaf = false;
            link = -1;
        }
    };
    vector<node> t;
    vector<vector<int>> ad;
    vector<int> st, en, mp;
    vector<int> cnt;
    int dt = 0;
    AC() {
        mp = vector<int> (1<<8, 0);
        for (char ch = 'a'; ch <= 'z'; ++ch) mp[ch] = ch - 'a
            ';
        t.resize(1);
        cnt.resize(1);
    }
    int add(string str, int i) {
        int ptr = 0;
        for (auto ch : str) {
            if (t[ptr].nxt[mp[ch]] < 0) {
                t[ptr].nxt[mp[ch]] = t.size();
                t.push_back(node(ptr, ch));
                cnt.push_back(0);
            }
            ptr = t[ptr].nxt[mp[ch]];
        }
        t[ptr].leaf = true;
        t[ptr].ind.push_back(i);
        return ptr;
    }
    int get_link(int v) {
        if (t[v].link == -1) {
            t[v].link = (!v || !t[v].par) ? 0 : go(get_link(t
                [v].par), t[v].pch);
        }
        return t[v].link;
    }
}

```

```

int go(int v, char ch) {
    if(t[v].nxt[mp[ch]] < 0) {
        t[v].nxt[mp[ch]] = !v ? 0 : go(get_link(v), ch);
    }
    return t[v].nxt[mp[ch]];
}

void dfs(int v) {
    st[v] = ++dt;
    for (int u : ad[v]) {
        dfs(u);
        cnt[v] += cnt[u];
    }
    en[v] = dt;
}

void calc() {
    int k = t.size();
    st.resize(k);
    en.resize(k);
    ad.resize(k);
    for (int i = 1; i < k; i++)
        ad[get_link(i)].push_back(i);
    dfs(0);
}

} cc;
int cs = 1;
void solve() {
    cc bc;
    int n; cin >> n;
    vector<int> res(n + 1, 0);
    string text;
    cin >> text;
    string p;
    for (int i = 1; i <= n; i++) {
        cin >> p;
        bc.add(p, i);
    }
    int vertex = 0;
    for (int i = 0; i < text.length(); i++) {
        vertex = bc.go(vertex, text[i]);
        bc.cnt[vertex]++;
    }
    bc.calc();
    for (int i = 1; i < bc.t.size(); i++) {
        for (int &x : bc.t[i].ind) res[x] += bc.cnt[i];
    }
    cout << "Case " << cs++ << ":\n";
    for (int i = 1; i <= n; i++) cout << res[i] << "\n";
}

```

7.2 Manacher

```

struct Manacher {
    vector<int> p[2];
    // p[1][i] = (max odd length palindrome centered at i) / 2
    // [floor division]
    // p[0][i] = same for even, it considers the right center
    // e.g. for s = "abbabba", p[1][3] = 3, p[0][2] = 2
    Manacher(string s) {
        int n = s.size();
        p[0].resize(n + 1);
        p[1].resize(n);
        for (int z = 0; z < 2; z++) {
            for (int i = 0, l = 0, r = 0; i < n; i++) {
                int t = r - i + !z;
                if (i < r) p[z][i] = min(t, p[z][l + t]);
                int L = i - p[z][i], R = i + p[z][i] - !z;
                while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
                    p[z][i]++, L--, R++;
                if (R > r) l = L, r = R;
            }
        }
    }
    bool is_palindrome(int l, int r) {
        int mid = (l + r + 1) / 2, len = r - l + 1;
        return 2 * p[len % 2][mid] + len % 2 >= len;
    }
};

int32_t main() {
    string s; cin >> s;
    Manacher M(s);
    int n = s.size();
    for (int i = 0; i < n; i++) {
        cout << 2 * M.p[1][i] + 1 << ' ';
        if (i + 1 < n) cout << 2 * M.p[0][i + 1] << ' ';
    }
    cout << '\n';
    return 0;
}

```

7.3 palindromitree

```

const int MAXN = 105000;
struct node {
    int next[26];
    int len;
    int sufflink;
    int num;
};

```

```

int len;
char s[MAXN];
node tree[MAXN];
int num; // node 1 - root with len -1, node 2 -
          // root with len 0
int suff; // max suffix palindrome
long long ans;
bool addLetter(int pos) {
    int cur = suff, curlen = 0;
    int let = s[pos] - 'a';
    while (true) {
        curlen = tree[cur].len;
        if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos])
            break;
        cur = tree[cur].sufflink;
    }
    if (tree[cur].next[let]) {
        suff = tree[cur].next[let];
        return false;
    }
    num++;
    suff = num;
    tree[num].len = tree[cur].len + 2;
    tree[cur].next[let] = num;
    if (tree[num].len == 1) {
        tree[num].sufflink = 2;
        tree[num].num = 1;
        return true;
    }
    while (true) {
        cur = tree[cur].sufflink;
        curlen = tree[cur].len;
        if (pos - 1 - curlen >= 0 && s[pos - 1 - curlen] == s[pos]) {
            tree[num].sufflink = tree[cur].next[let];
            break;
        }
    }
    tree[num].num = 1 + tree[tree[num].sufflink].num;
    return true;
}

void initTree() {
    num = 2; suff = 2;
    tree[1].len = -1; tree[1].sufflink = 1;
    tree[2].len = 0; tree[2].sufflink = 1;
}

int main() {
    //assert(freopen("input.txt", "r", stdin));
    //assert(freopen("output.txt", "w", stdout));
}

```

```

gets(s); len = strlen(s);
initTree();
for (int i = 0; i < len; i++) {
    addLetter(i);
    ans += tree[suff].num;
}
}

```

7.4 suffixArray

```

struct SuffixArray {
    int n; string s;
    vector<int> sa, rankv, tmp, lcp;
    SuffixArray(const string &str) {
        s = str; n = s.size(); sa.resize(n); rankv.resize(n);
        tmp.resize(n); build(); build_lcp();
    }
    void counting_sort(int k) {
        int maxi = max(256ll, (long long)n) + 5;
        vector<int> cnt(maxi, 0), sa2(n);
        for (int i = 0; i < n; i++) {
            int key = (i + k < n) ? rankv[i + k] + 1 : 0;
            cnt[key]++;
        }
        for (int i = 1; i < maxi; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) {
            int key = (sa[i] + k < n) ? rankv[sa[i] + k] + 1 : 0;
            sa2[--cnt[key]] = sa[i];
        }
        sa.swap(sa2);
    }
    void build() {
        for (int i = 0; i < n; i++) {
            sa[i] = i; rankv[i] = (unsigned char)s[i];
        }
        for (int k = 1; k < n; k <= 1) {
            counting_sort(k);
            counting_sort(0);
            tmp[sa[0]] = 0;
            int r = 0;
            for (int i = 1; i < n; i++) {
                int cur1 = rankv[sa[i]], cur2 = (sa[i] + k < n
                    ? rankv[sa[i] + k] : -1);
                int prev1 = rankv[sa[i - 1]], prev2 = (sa[i - 1]
                    + k < n ? rankv[sa[i - 1] + k] : -1);
                if (cur1 != prev1 || cur2 != prev2) r++;
            }
        }
    }
}

```

```

        tmp[sa[i]] = r;
    }
    rankv.swap(tmp);
    if (r == n - 1) break;
}
}
void build_lcp() {
    lcp.assign(max(0, n - 1), 0);
    vector<int> inv(n);
    for (int i = 0; i < n; ++i) inv[sa[i]] = i;
    int k = 0;
    for (int i = 0; i < n; ++i) {
        if (inv[i] == n - 1) { k = 0; continue; }
        int j = sa[inv[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j
            + k]) ++k;
        lcp[inv[i]] = k;
        if (k) --k;
    }
}
};
void solve(const string &s, const string &t) {
    // build local combined string so we don't mutate inputs
    int n = s.size();
    string combined = s + '$' + t;
    int N = combined.size();
    SuffixArray sa(combined);
}

```

7.5 z kmp

```

vector<int> prefix_function(string s) {
    int n = (int)s.length();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j])
            j = pi[j - 1];
        if (s[i] == s[j])
            j++;
        pi[i] = j;
    }
    return pi;
}
vector<int> z_function(string pattern, string text) {
    string s = pattern + "$" + text;
    int n = size(s);
}

```

```

vector<int> z(n);
for (int i = 1, l = 0, r = 0; i < n; ++i) {
    if (i <= r)
        z[i] = min(r - i + 1, z[i - l]);
    while (i + z[i] < n && s[z[i]] == s[i + z[i]])
        ++z[i];
    if (i + z[i] - 1 > r)
        l = i, r = i + z[i] - 1;
}
return z;
}

```

8 sublime build

```

{
    "cmd" : ["g++ -std=c++20 $file_name -o $file_base_name &&
        timeout 5s ./$file_base_name<input.txt>output.txt"],
    "selector" : "source.cpp",
    "shell": true
    //"working_dir" : "$file_path"
}

```

9 template

```

#include<bits/stdc++.h>
using namespace std;
#define endl '\n'
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define int long long
typedef tree<int, null_type, less<int>, rb_tree_tag,
    tree_order_statistics_node_update> o_set;
typedef tree<int, null_type, less_equal<int>, rb_tree_tag,
    tree_order_statistics_node_update> o_multiset;
mt19937 rng(chrono::steady_clock::now().time_since_epoch().
    count());
void solve() { //order_of_key, find_by_order }
int32_t main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int tc = 1; cin >> tc;
    for (int i = 1; i <= tc; i++) { solve(); } return 0;
}

```